UNIVERSITÀ DEGLI STUDI DI MODENA E REGGIO EMILIA

DIPARTIMENTO DI INGEGNERIA "ENZO FERRARI"

Laurea Triennale in Ingegneria Informatica

Augmented Reality Framework: AR Foundation

Relatore:

Prof: Nicola Bicocchi

Candidato:

Manuel Gherardi

Anno Accademico 2025/2026

Indice

1 Introduzione **5**

2 Realtà Aumentata **6**
2.1 Che cos'è la Realtà Aumentata? 6
2.2 Pro e Contro 10
2.2.1 Pro 10
2.2.2 Contro 11
2.3 Come funziona? 13
2.4 Tipologie di Realtà Aumentata 17
2.4.1 AR a Riconoscimento Marker 17
2.4.2 Markerless AR 17
2.5 AR vs VR vs XR vs MR vs Spatial Computing 17

3 Unity **19**
3.1 Pilastri di Unity 19
3.2 Camera 19
3.3 Sistema di coordinate 20
3.4 Prefab e istanziazione 20
3.5 Raycasting 20

4 AR Foundation **21**
4.0.1 Architettura 24
4.0.2 Componenti Principali 27
4.0.3 Perché AR Foundation? 28

5 App **30**
5.1 Analisi Dei Requisiti 31
5.1.1 Scopo e Uso 31
5.1.2 Requisiti Funzionali 33
5.1.3 Requisiti Non Funzionali 34
5.1.4 UML Diagrams 35
5.2 Design Pattern 38
5.2.1 Convenzioni di Nomenclatura 38
5.2.2 Command 38

5.2.3 State 39
5.2.4 Observer 39
5.2.5 Factory Method 40
5.2.6 Struttura Dati: CommandHistory 41
5.3 Analisi delle Prestazioni e Profiling 42
5.3.1 Tipologie di Grafici e Metodologia di Lettura 43
5.3.2 Testing 44
5.3.3 Limiti della Sperimentazione 49

6 Conclusioni **50**

Elenco delle figure

2.1 Scene of Iron Man 2 (2010)[21] 7
2.2 Google Glass, 2013 [20] 8
2.3 An example of AR videogame: Invizimals, game of PSP [19] 8
2.4 AR General Working Architecture[23] 13
2.5 AR Pipeline 14
2.6 SLAM [22] 15
2.7 The Reality-Virtuality Continuum according to Milgram [8] 18

4.1 AR Foundation Workflow (Example Feature: Image Tracking) [11] . . 24
4.2 AR Foundation Architecture vs Native Architecture [24] 28

5.1 Screenshots while my app is running on my phone 30
5.2 Screen while my app is running in Unity XR Simulator 31
5.3 Activity Diagram, Focus: Generale 35
5.4 Activity Diagram, Focus: Selezione e Drag 36
5.5 Activity Diagram, Focus: GUI 36
5.6 Activity Diagram, Focus: GUI Delete e Undo 37
5.7 Class Diagram, Focus: Factory e Observer 37
5.8 Class Diagram, Focus: State 37
5.9 Class Diagram, Focus: Command 38
5.10 Schermata generale dell'Unity Profiler durante l'esecuzione dell'applicazione sul dispositivo di test. 42
5.11 Scheda *Rendering* dell'Unity Profiler. 45
5.12 Scheda *Memory* dell'Unity Profiler. 46
5.13 Scheda *Selection Highlights* dell'Unity Profiler. 47
5.14 Scheda *Capture Highlights* dell'Unity Profiler. 47

Capitolo 1

Introduzione

Negli ultimi anni, la Realtà Aumentata (AR) ha completato la sua transizione da tecnologia sperimentale a strumento pervasivo, capace di ridefinire l'interazione uomo-macchina in settori che spaziano dall'industria 4.0 alla medicina, fino all'istruzione e all'intrattenimento. A differenza della Realtà Virtuale, che isola l'utente in un ambiente digitale, l'AR arricchisce la percezione del mondo fisico sovrapponendovi elementi virtuali in tempo reale e in modo geometricamente coerente.Nonostante il potenziale rivoluzionario, la diffusione di massa delle applicazioni AR ha dovuto affrontare una barriera significativa: la frammentazione hardware e software. Per lungo tempo, lo sviluppo di soluzioni AR ha richiesto l'utilizzo di SDK(Software Development Kit) nativi differenti, come Apple ARKit per i dispositivi iOS e Google ARCore per l'ecosistema Android. Questa dicotomia ha storicamente comportato un aumento dei costi di produzione, problemi di portabilità del codice e una complessa gestione degli aggiornamenti.In questo panorama si colloca AR Foundation, il framework cross-platform sviluppato da Unity. AR Foundation si propone come uno strato di astrazione software che unifica le API dei principali sistemi operativi, permettendo agli sviluppatori di progettare un'architettura logica universale ("write once, run anywhere"). Attraverso l'uso di componenti modulari (come i Manager per il tracciamento delle superfici o dei volti), il framework non solo ottimizza il flusso di lavoro, ma garantisce l'accesso alle funzionalità hardware più avanzate dei dispositivi moderni senza perdere in prestazioni. In questo lavoro, risultato dell'attivitá progettuale svolta presso il dipartimento di Ingegneria "Enzo Ferrari" dell'Universitá di Modena e Reggio Emilia, l'obiettivo è analizzare l'architettura e le potenzialità di AR Foundation come soluzione al problema della frammentazione nello sviluppo di applicazioni in Realtà Aumentata. Nella prima parte della tesi, a carattere compilativo, viene fornito un quadro comparativo dettagliato tra AR Foundation e gli SDK nativi sottostanti ARCore e ARKit, valutando l'impatto dell'astrazione in termini di funzionalità supportate, flessibilità architetturale e prestazioni. Successivamente, la trattazione si focalizzerà su una fase sperimentale, incentrata sulla progettazione e sull'implementazione di un'applicazione dimostrativa, al fine di quantificare sul campo l'efficienza del framework nei principali casi d'uso industriali o applicativi.

Capitolo 2

Realtà Aumentata

2.1 Che cos'è la Realtà Aumentata?

Per **realtà aumentata(AR, oppure Augmented Reality (RA))** [1][2] si intende una tecnologia di computer grafica che integra in tempo reale informazioni digitali nell'ambiente reale dell'utente, arricchendone la percezione sensoriale con contenuti che i cinque sensi, da soli, non potrebbero cogliere. L'integrazione non comporta necessariamente un'aggiunta di informazioni: può consistere anche nella loro riduzione, quando ciò permette di rappresentare una situazione in modo più chiaro e utile. Gli elementi aumentati possono essere resi disponibili tramite dispositivi mobili come gli smartphone, tramite un PC dotato di webcam o di altri sensori, oppure tramite dispositivi indossabili di visione (ad es. occhiali a proiezione retinica), di ascolto (auricolari) e di manipolazione (guanti).

Storicamente[3],le prime esperienze di realtà virtuale e aumentata risalgono agli anni '60. Nel 1962 Morton Heilig realizzò il Sensorama, una cabina che offriva un'esperienza multisensoriale con filmato stereoscopico, audio, vibrazioni, vento e odori, ma senza alcuna interattività. Nel 1968 Ivan Sutherland, il "padre della computer grafica", con il suo studente Bob Sproull, sviluppò il primo head-mounted display, noto come "Spada di Damocle", poichè era così pesante da dover essere sostenuto da un braccio meccanico attaccato al soffitto: mostrava grafiche wireframe tridimensionali, grazie a due tubi catodici, che seguivano i movimenti della testa, sovrapposte all'ambiente reale grazie a lenti semitrasparenti, e ne fanno quindi anche il primo sistema di AR. Il termine Augmented Reality fu coniato all'inizio degli anni '90 da Thomas Caudell e David Mizell, ricercatori della Boeing, che lo usarono per descrivere un display indossabile capace di sovrapporre al campo visivo dei tecnici le istruzioni digitali per il cablaggio degli aerei. Nel 1994 Paul Milgram e Fumio Kishino introdussero il continuum realtà-virtualità, che va dall'ambiente reale a quello completamente virtuale. Tra i due estremi si collocano l'Augmented Reality, prevalentemente reale con elementi virtuali aggiunti, e l'Augmented Virtuality, prevalentemente virtuale con elementi reali; l'intera fascia intermedia prende il nome

di Mixed Reality. Infine, dalla metà degli anni 2010 si è diffuso il termine Extended Reality (XR), che non ha un singolo autore ma raggruppa VR, AR e MR. Proprio in contemporanea all'AR, nel 1988 Mark Weiser [4][5][6], Chief Technologist presso il famoso centro di ricerca Xerox PARC(Palo Alto Research Center), formulò il concetto di **Ubiquitous Computing** (informatica ubiqua o pervasiva): una visione in cui i computer e le tecnologie associate diventano invisibili, fino a confondersi con la vita quotidiana. Questa invisibilità si realizza attraverso la "virtualità incarnata" (embodied virtuality), ovvero l'integrazione dei computer nel mondo fisico. Un'automobile moderna, ad esempio, può essere guidata in modo efficiente senza comprendere il funzionamento dei numerosi motori, attuatori e sensori che contiene: Weiser estese questa idea alle infrastrutture di calcolo e comunicazione, sintetizzandola così: `"Le tecnologie più profonde sono quelle che scompaiono. Si intrecciano nella trama della vita quotidiana fino a diventarne indistinguibili"`[6]

L'AR si divide in due tipi[8]:

- **Spatial AR**, Proietta la luce direttamente sull'ambiente reale (usando video proiettori fisici fissati nella stanza). Non richiede visori o smartphone per essere vista. Un esempio tipico è il spatial projection mapping (es. proiettare la trama di un tessuto su un divano reale). Consiste proprio nel proiettare interfacce o elementi digitali su superfici reali (tavoli, pareti, oggetti) e rendere quelle superfici interattive.



Figura 2.1: Scene of Iron Man 2 (2010)[21]

- **See-through AR**, in cui si fa uso di periferiche o dispositivi indossabili o di semplice visualizzazione come visore o schermi.
  - **Optical See-through**, Vetri o lenti trasparenti (come i Google Glass o i Magic Leap) in cui la luce reale passa direttamente e le immagini digitali vengono proiettate sopra, quindi vedi il mondo reale direttamente con i tuoi occhi e la grafica vi viene sovrapposta tramite prismi/specchi.

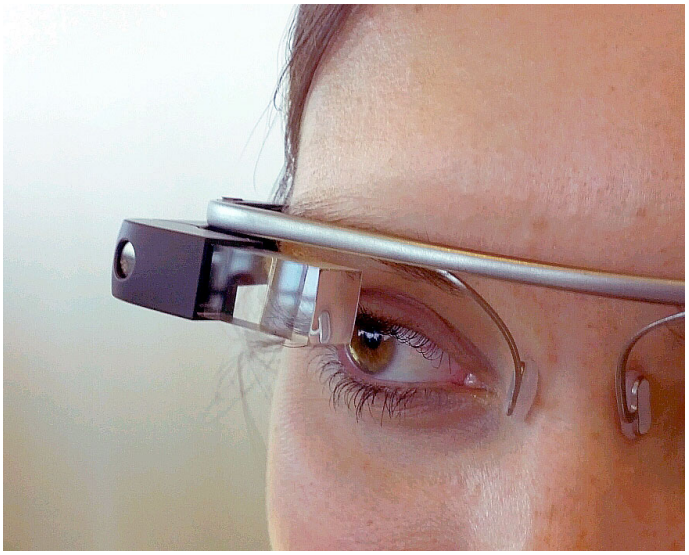


Figura 2.2: Google Glass, 2013 [20]

– **Video See-through (Passthrough)**, che catturano il mondo reale tramite telecamere e lo riproiettano su schermi interni ad altissima risoluzione insieme agli elementi digitali. Visori chiusi (es. Apple Vision Pro, Meta Quest 3) o dispositivo con camera (es. Invizimals su PSP), dove microcamere riprendono l'esterno e mostrano il video su uno schermo insieme alla grafica 3D.



Figura 2.3: An example of AR videogame: Invizimals, game of PSP [19]

Dalla definizione precedentemente scritta, si può dedurre che l'AR è adatta ad ogni ambito[1]:

- **Medicina e Sanità**
  in medicina e sanità si introduce per migliorare la chirurgia assistita, ovvero tramite sovrapposizione di scansioni (TAC, risonanze) direttamente sul corpo

del paziente durante gli interventi per guidare l'incisione e la precisione del chirurgo, e sempre in questo ambito, si usa per la formazione di medici mediante simulazioni delle procedure mediche e anatomiche su modelli virtuali interattivi senza rischi per i pazienti;

- **Industria e Luogo Lavorativo**
  nell'industria e in ambito lavorativo in generale viene introdotta nella manutenzione e assemblaggio, mediante istruzioni di montaggio o schemi tecnici sovrapposti in tempo reale sui macchinari per guidare i tecnici a mani libere (remote assistance), e anche sulla sicurezza, in cui consiste nella segnalazione visiva di pericoli o aree ad alto rischio direttamente nel campo visivo degli operatori;

- **Architettura, Progettazione e Pianificazione Urbana**
  in architettura, progettazione e pianificazione Urbana viene usata nella visualizzazione dei progetti, per il rendering 3D di edifici o strutture sovrapposti sul terreno reale prima dell'inizio dei lavori, oppure per l'ispezione di interni, offrendo la possibilità di guardare "attraverso le pareti" per individuare tubature, cablaggi o elementi strutturali interni;

- **Archeologia, Cultura e Turismo**
  In archeologia, cultura e turismo viene impiegato nella ricostruzione di rovine, nella visualizzazione in tempo reale di come apparivano monumenti ed edifici antichi direttamente sopra i siti archeologici attuali, e nelle guide ed esperienze di musei, tramite punti di interesse interattivi e contenuti multimediali sovrapposti alle opere o alle attrazioni turistiche della città;

- **Commercio, Marketing e Retail**
  nel commercio, marketing e retail, con la famosa filosofia del "Try Before You Buy" (Prova prima di acquistare): con la visualizzazione di mobili e oggetti d'arredo nell'ambiente di casa prima dell'acquisto, oppure prova virtuale di capi d'abbigliamento, occhiali e cosmetici, e con l'Augmented Advertising, che consiste in campagne pubblicitarie interattive accessibili inquadrando imballaggi, riviste o cartelloni;

- **Istruzione e Formazione (STEM)**
  nell'istruzione e formazione (STEM) tramite libri e materiali interattivi, ovvero con l'integrazione dei libri di testo classici con modelli 3D animati di molecole, organi o corpi celesti e in ambito più pratico con l'addestramento al volo e militare tramite display o visori per piloti (Head-Up Display o HUD) con dati tattici, rotte e parametri di volo sovrapposti al parabrezza;

- **Automotive e Navigazione**
  nell'automotive e navigazione sempre usando l'implementazione precedente, ovvero tramite Head-Up Display (HUD) proiettando i dati di navigazione, velocità e segnali stradali direttamente sul parabrezza dell'auto, e nella navigazione

pedonale o per veicoli, tramite indicazioni stradali visive e frecce dinamiche sovrapposte direttamente sulla strada inquadrata dalla fotocamera;

- **Intrattenimento e Videogiochi**
  nell'intrattenimento e nei videogiochi, in cui consiste nel gaming geolocalizzato, ovvero videogiochi che trasformano l'ambiente cittadino reale in un campo da gioco interattivo (es. Pokemon GO);

- **Eventi dal Vivo e Trasmissioni**
  infine negli eventi dal vivo e trasmissioni, mediante grafiche e statistiche in tempo reale sovrapposte ai campi durante gli eventi sportivi.

2.2 Pro e Contro

In tutti questi ambiti, possiamo studiare e analizzare come l'AR si comporta sotto diversi aspetti e stilare una lista di caratteristiche che può portare questa tecnologia, sia positive sia negative.

2.2.1 Pro

1. **Continuità con la realtà e sicurezza d'uso**
   A differenza della realtà virtuale (VR), che isola completamente l'utente portandolo in un mondo artificiale, la realtà aumentata mantiene la persona ancorata all'ambiente fisico reale. Questo offre vantaggi cruciali:

   - **Consapevolezza dell'ambiente circostante**
     L'utente continua a vedere dove cammina o lavora, riducendo rischi di collisione o disorientamento.
   - **Minore affaticamento visivo/sensoriale**
     Non dovendo ricreare un intero mondo 3D ma solo "strati" di informazione, l'esperienza risulta spesso più naturale e meno alienante.

2. **Miglioramento della precisione e riduzione degli errori umani**
   La sovrapposizione visiva di dati e modelli direttamente sull'oggetto o sul corpo su cui si sta lavorando elimina la necessità di consultare continuamente manuali o schermi esterni:

   - **Minori tassi di errore**
     In ambiti critici come la chirurgia o la manutenzione industriale, le guide visive passo-passo riducono le dimenticanze e gli sbagli operativi.
   - **Aumento della velocità di esecuzione**
     L'operatore o il tecnico ha le mani libere e le informazioni visive posizionate esattamente dove servono.

3. **Abbattimento dei costi e ottimizzazione dei processi**

   - **Prototipazione e visualizzazione anticipata**
     Architetti, designer e pianificatori possono testare un progetto nell'ambiente reale prima di costruire o produrre fisicamente il bene, riducendo lo spreco di materiali e correggendo difetti in anticipo.
   - **Riduzione dei resi nel commercio**
     La possibilità di provare virtualmente mobili o vestiti prima dell'acquisto ("Try Before You Buy") aumenta la certezza del consumatore e abbassa la quota di resi.

4. **Esperienza di apprendimento più intuitiva e coinvolgente**

   - **Apprendimento con l'esperienza**
     Sia nell'istruzione scolastica (STEM) sia nella formazione aziendale, visualizzare ed interagire con oggetti 3D (come molecole, motori o organi anatomici) facilita la comprensione di concetti complessi rispetto alla sola lettura.
   - **Accessibilità del sapere**
     Le informazioni guida o storiche vengono erogate in modo contestuale (es. inquadrando un monumento o una rovina), rendendo l'esplorazione autonoma e ricca di valore culturale.

5. **Collaborazione e assistenza a distanza**

   - **Remote Assistance**
     Esperti qualificati possono guidare tecnici o utenti sul campo disegnano indicazioni visive direttamente nello schermo di chi opera, eliminando tempi e costi di trasferta.
   - **Lavoro di squadra facilitato**
     Più persone possono visualizzare e manipolare simultaneamente lo stesso modello 3D nello spazio reale.

2.2.2 Contro

Nonostante i numerosi vantaggi, la diffusione della realtà aumentata comporta diversi rischi, pericoli e svantaggi che spaziano dalla sicurezza fisica alle implicazioni etiche e sociali.

1. **Rischi per la sicurezza fisica e distrazione**

- **Saturazione cognitiva e perdita di attenzione**
  L'accumulo di informazioni visive sovrapposte al mondo reale può sovraccaricare il cervello, riducendo la prontezza di riflessi in situazioni critiche (come la guida di veicoli o la gestione di macchinari)
- **Incidenti da distrazione**
  L'uso continuo di visori o smartphone per interagire con contenuti AR nell'ambiente urbano aumenta il rischio di cadute, attraversamenti involontari o collisioni con ostacoli e persone.

2. **Privacy e sorveglianza invasiva**

- **Raccolta costante di dati ambientali e personali**
  Per funzionare, i dispositivi AR utilizzano fotocamere e sensori sempre attivi che mappano continuamente l'ambiente circostante, registrando volti di passanti, targhe ed spazi privati senza consenso esplicito.
- **Tracciamento dei comportamenti e biometria**
  I visori avanzati monitorano lo sguardo (eye tracking), i movimenti del corpo e le reazioni emotive, fornendo alle aziende dati estremamente sensibili sulle preferenze e sull'attenzione degli utenti.

3. **Sicurezza informatica e manipolazione della realtà**

- **Hacking ed estorsione informativa**
  Un attacco informatico potrebbe iniettare informazioni false nel campo visivo dell'utente (ad esempio alterare i segnali stradali su un display guida o nascondere un ostacolo reale), con conseguenze potenzialmente fatali, portando quindi a danni irreversibili o comunque molto gravi ad ambiente, beni o addirittura persone.
- **Falsificazione del contesto (Reality Hacking)**
  Sovrapporre elementi grafici non autorizzati o ingannevoli su luoghi fisici (come pubblicità abusive, messaggi diffamatori o informazioni manipolate) può alterare la percezione della realtà da parte degli utenti.

4. **Impatto Psicologico e Dipendenza**
   Rischi in comune con i Social Network:

- **Sovraccarico sensoriale e alienazione**
  La costante sovrapposizione di notifiche e contenuti digitali al mondo fisico rende difficile la disconnessione, aumentando i livelli di stress e ansia.
- Un utilizzo incontrollato può portare a preferire l'ambiente arricchito dai filtri digitali rispetto al mondo reale, condizionando negativamente la socializzazione e le relazioni interpersonali dirette.

5. **Limiti Tecnici ed Economici**

- **Costi elevati dei dispositivi**
  I visori e gli strumenti AR di livello professionale mantengono prezzi accessibili solo a grandi aziende o settori specializzati, rischiando di ampliare il divario digitale (digital divide)
- **Consumo energetico e limiti hardware**
  I sensori e i processori AR richiedono un'elevata potenza di calcolo, comportando una durata limitata della batteria e problemi di surriscaldamento nei dispositivi portatili.

2.3 Come funziona?

Architettura

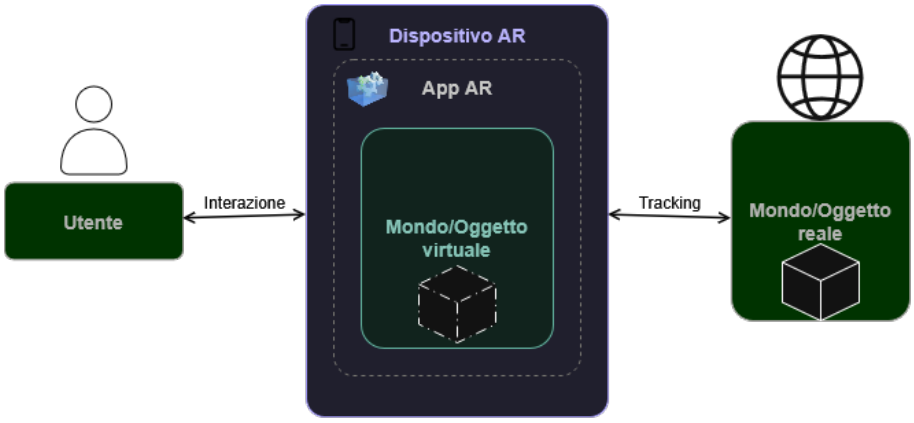


Figura 2.4: AR General Working Architecture[23]

Il funzionamento di un sistema di AR si basa su un ciclo continuo di elaborazione (loop di feedback) suddiviso in tre fasi sequenziali fondamentali[2]:



Figura 2.5: AR Pipeline

- **Sensazione e Tracciamento**
  Per fare in modo che gli elementi virtuali possano essere inseriti nel mondo reale in modo credibile, uno smartphone o un visore devono comprendere in tempo reale e con estrema precisione dove si trovano nello spazio tridimensionale. Per ottenere questo risultato, il dispositivo si affida all'acquisizione continua di flussi video e dati fisici attraverso una combinazione di sensori ottici e inerziali. Tra i sensori ottici troviamo le fotocamere tradizionali, che riprendono l'ambiente circostante, e i sensori di profondità avanzati come il LiDAR, che emettono impulsi di luce invisibile per calcolare la distanza esatta degli oggetti e creare una mappa tridimensionale millimetrica dello spazio. A questi si affiancano l'Unità di Misura Inerziale, comunemente nota come IMU, che integra accelerometri per misurare le variazioni di velocità e giroscopi per tracciare l'orientamento e la velocità di rotazione. Grazie a questa componentistica, il sistema è in grado di captare anche i movimenti minimi o rapidi della testa o delle mani dell'utente, compensando i ritardi visivi e garantendo la fluidità dell'esperienza.

  L'aspetto più importante di questa fase è la cosiddetta stima della posa, o Pose Estimation, che calcola la posizione e l'orientamento del dispositivo in sei gradi di libertà, noti come 6DoF. Questi sei gradi descrivono tutte le possibili modalità di movimento di un corpo nello spazio: tre riguardano la traslazione lineare lungo gli assi cartesiani, ovvero lo spostamento in avanti e indietro, a sinistra e a destra, e in alto e basso; gli altri tre riguardano la rotazione, specificatamente l'inclinazione in avanti e indietro nota come pitch, la rotazione orizzontale a destra e sinistra nota come yaw, e l'inclinazione laterale nota come roll. Per calcolare questi dati e mappare l'ambiente, i dispositivi utilizzano principalmente due strade. La prima è il tracciamento basato su marcatori, o Marker-based, in cui il software riconosce simboli o pattern geometrici pre-

stampati, come i codici QR; conoscendo le dimensioni e la posizione esatta del marcatore nel mondo reale, il sistema sa immediatamente dove posizionare il contenuto virtuale. La seconda tecnica, molto più avanzata e utilizzata negli ambienti aperti o non preparati, è la localizzazione e mappatura simultanee, nota come SLAM. Questa tecnologia consente al dispositivo di costruire progressivamente una mappa 3D dell'ambiente circostante e, nello stesso identico momento, di calcolare la propria posizione esatta all'interno di quella mappa senza bisogno di alcun punto di riferimento esterno, permettendo all'utente di muoversi liberamente.

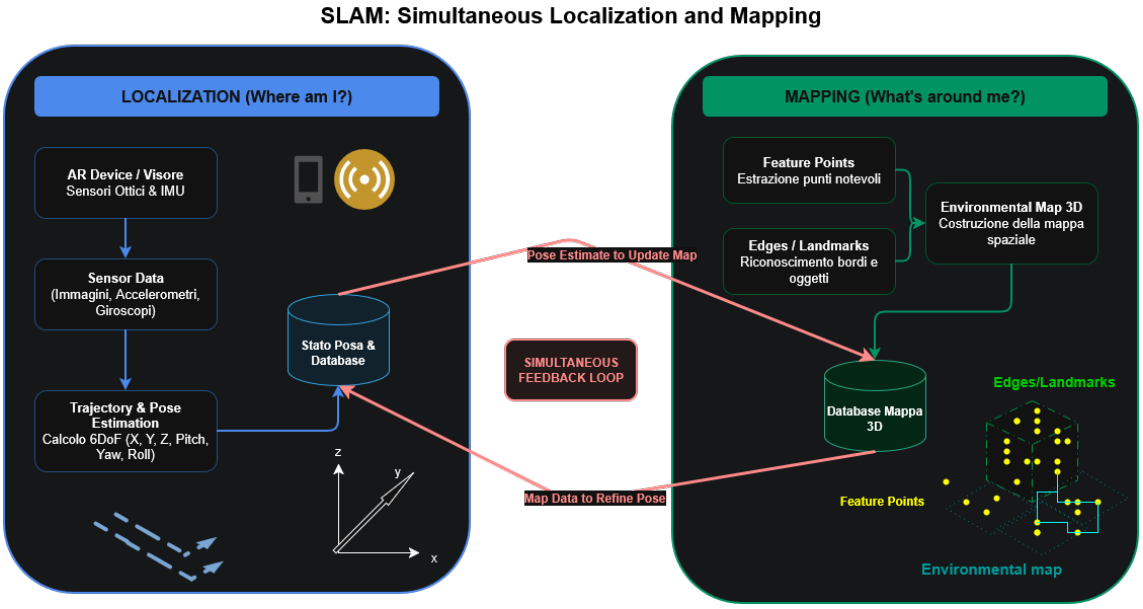


Figura 2.6: SLAM [22]

- **Elaborazione e Riconoscimento**
  Una volta che i dati grezzi sono stati raccolti dai sensori e dalle fotocamere, il software li interpreta e cerca di dare un significato logico e semantico a ciò che la telecamera sta inquadrando. Questo processo si fonda sui principi della Computer Vision, ovvero la branca dell'intelligenza artificiale che permette ai computer di "vedere" e comprendere le immagini e i video digitali, analizzando i singoli pixel per riconoscere bordi, forme, angoli e variazioni di luce. Per fare un salto di qualità nell'interpretazione dell'ambiente, gli algoritmi si affidano a modelli di intelligenza artificiale complessi, come le reti neurali convoluzionali, note anche come CNN. Queste reti di apprendimento profondo sono particolarmente abili nell'elaborazione visiva e consentono al sistema di classificare gli oggetti con precisione, distinguendo ad esempio un tavolo da una sedia, una parete da una porta, oppure riconoscendo un componente meccanico specifico all'interno di un'officina.

Attraverso questo processo, il software riesce a riconoscere i piani e le superfici fisiche presenti nell'ambiente, elaborando la nuvola di punti creata nella fase precedente per identificare le superfici orizzontali, come pavimenti e tavoli, e quelle verticali, come le pareti. Questa capacità è fondamentale perché permette agli oggetti virtuali di "posarsi" in modo logico sul mondo reale, come fare in modo che una sedia olografica poggi saldamente sul pavimento di una stanza. In contesti industriali o smart factories particolarmente avanzati, questa fase di riconoscimento si spinge oltre, prevedendo l'interconnessione con un Digital Twin, ovvero un gemello digitale memorizzato sul cloud. Quando il sistema inquadra un macchinario reale, non si limita a riconoscerne la forma geometrica nello spazio, ma lo associa alla sua controparte virtuale dinamica. Questo collegamento permette di sincronizzare in tempo reale i dati telemetrici, come la pressione, la temperatura e i cicli di lavoro, e lo stato di salute dell'impianto provenienti dai sensori IoT, ovvero gli oggetti intelligenti connessi a Internet, consentendo all'operatore di visualizzare grafici, schemi e avvisi diagnostici direttamente sovrapposti al macchinario che sta guardando.

- **Rendering e Visualizzazione**
  Infine il software si occupa di generare concretamente l'immagine finale che l'utente vedrà attraverso lo schermo del dispositivo, fondendo in modo fluido, coerente e in tempo reale il mondo reale con quello virtuale. A occuparsi di questo compito è il motore grafico, o Game Engine, un software potente originariamente sviluppato per l'industria dei videogiochi ma oggi indispensabile in AR, che si incarica di calcolare la geometria dei modelli tridimensionali, applicare le texture, cioè i materiali visivi e i colori, gestire le animazioni e simulare le leggi della fisica sugli oggetti olografici. Per fare in modo che l'illusione ottica sia perfetta, il motore grafico applica una matrice di proiezione, ovvero un calcolo matematico basato sulla posizione esatta del visore rilevata nella prima fase di tracciamento. Questo calcolo deforma e scala gli oggetti 3D rispettando le regole della prospettiva umana, facendo in modo che gli elementi virtuali vicini appaiano più grandi e quelli lontani più piccoli.

La coerenza visiva viene mantenuta grazie a due elementi cruciali: l'illuminazione coerente e l'occlusione. Nel primo caso, il sistema analizza le fonti di luce presenti nell'ambiente reale e illumina di conseguenza l'oggetto virtuale, generando ombre realistiche che ne impediscono l'effetto "piatto" o posticcio. Nel secondo caso, l'occlusione risolve il problema della profondità gestendo le sovrapposizioni: se una persona passa la mano o un oggetto reale davanti a un ologramma, la mano deve coprire l'oggetto virtuale perché si trova più vicina agli occhi rispetto ad esso. Senza l'occlusione, l'ologramma rimarrebbe sempre visibile in primo piano, rovinando completamente il senso di immersione. Infine, il ciclo si completa gestendo l'interazione dell'utente: attraverso gesture, ossia i movimenti delle mani tracciati dalle telecamere, comandi vocali o input

touch, l'utente può manipolare l'oggetto virtuale, spostandolo, ridimensionandolo o aprendolo, oppure può inviare comandi diretti che modificano lo stato del mondo reale o dei sistemi fisici connessi.

2.4 Tipologie di Realtà Aumentata

Ne esistono due tipi: AR a riconoscimento Marker,la quale è più economica e accessibile, e Markerless AR, la quale offre un coinvolgimento maggiore.

2.4.1 AR a Riconoscimento Marker

Con questa tipologia, affinchè avvenga l'implementazione dell'AR, è necessario un trigger fisico (Marker) nell'ambiente reale, ad esempio un codice QR, un'immagine o un altro marker di destinazione. Quando la fotocamera di un dispositivo rileva il marker, attiva la visualizzazione dell'esperienza AR associata. Poiché questo tipo di AR è accessibile in qualsiasi momento da una serie di dispositivi, è quello più flessibile.

2.4.2 Markerless AR

Con questa tipologia, invece, non c'è alcun bisogno di un trigger fisico, poichè si basa direttamente su sensori, GPS, accelerometri e fotocamere del dispositivo per mappare in tempo reale l'ambiente circostante dell'utente. Questo mapping avviene spesso grazie ad algoritmi e alla computer vision, i quali determinano dove posizionare i contenuti digitali, rendendo l'esperienza più spontanea e dinamica.

2.5 AR vs VR vs XR vs MR vs Spatial Computing

Ora vedremo la differenza tra queste tecnologie molto simili ma diverse tra loro[7][8]:

- **Virtual Reality (VR)**
  Permette la creazione e l'immersione in un ambiente virtuale 3D, consentendo all'utente di muoversi liberamente e di interagire con esso. È possibile grazie ad elaboratori e interfacce dedicate, come visori, guanti dati (*wired gloves*), cybertute e piattaforme omnidirezionali. Viene utilizzata per simulazioni, videogiochi e addestramento; un esempio diffuso è la serie di visori Meta Quest. Esistono due tipi:
  - **Semi-Immersive VR**, nel quale i dispositivi VR rappresentano in modo preciso e reale una parte del punto di vista dell'utente
  - **Immersive VR**, dove i dispositivi rappresentano l'intero punto di vista dell'utente.

- **Spatial Computing**
  È il paradigma tecnologico che consente ai sistemi informatici di comprendere, mappare e interagire con lo spazio fisico 3D così come fanno gli esseri umani. Permette ai contenuti digitali di "vivere" nell'ambiente reale con persistenza spaziale (ad esempio, posizionare un quadro digitale su una parete reale e ritrovarlo esattamente nella stessa posizione al successivo utilizzo del visore). Lo Spatial Computing fornisce le basi tecnologiche necessarie per abilitare la Mixed Reality.

- **Mixed Reality (MR)**
  Fonde il mondo fisico e gli oggetti digitali, permettendo un'interazione dinamica in tempo reale tra elementi reali e virtuali (ad esempio, una palla virtuale che rimbalza sul pavimento reale di una stanza), che sfrutta lo Spatial Computing. Un esempio di dispositivo MR avanzato è l'Apple Vision Pro.

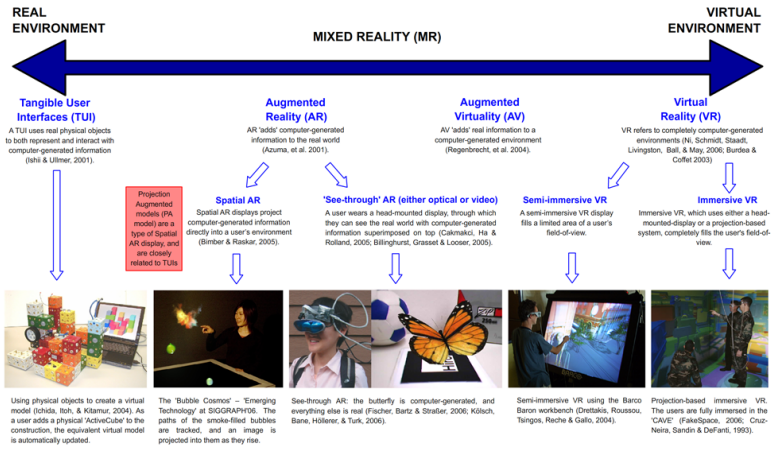


Figura 2.7: The Reality-Virtuality Continuum according to Milgram [8]

dove AV è Augmented Virtuality, una tecnologia di MR in cui l'ambiente generato al computer viene arricchito da oggetti del mondo fisico.

- **Extended Reality (XR)**
  Termine che racchiude VR, AR e MR. Quando un dispositivo viene definito XR (come Apple Vision Pro o Galaxy XR), significa che è in grado di passare fluidamente tra diverse modalità di realtà immersiva.

  In questo ambito rientra anche la **Augmented Reality (AR)** come detto in precedenza: l'integrazione in tempo reale di informazioni digitali nell'ambiente reale dell'utente, percepibili tramite dispositivi come smartphone, visori o auricolari. Tale integrazione può operare sia per addizione di elementi multimediali, sia per sottrazione di informazioni superflue (diminuzione della realtà), al fine di rendere la percezione dell'ambiente più chiara e funzionale.

Capitolo 3

Unity

Unity è un motore di gioco (*game engine*)[15][16][17] [18] multipiattaforma che consente lo sviluppo di videogiochi e altri contenuti interattivi, come visualizzazioni architettoniche o animazioni 3D in tempo reale. In questo capitolo introduco i concetti di Unity su cui si basa AR Foundation, necessari per comprendere il funzionamento dell'intero sistema.

3.1 Pilastri di Unity

Unity si basa su tre concetti. Il **GameObject** è l'entità di base della scena (un oggetto 3D, una luce, la camera): di per sé è un contenitore vuoto, il cui comportamento è definito dai *componenti* associati. Il **Transform** è il componente presente in ogni GameObject e gestisce posizione (x, y, z), rotazione e scala nello spazio 3D della scena. I **MonoBehaviour** sono gli script C# che si agganciano a un GameObject come componenti; ne fanno parte anche quelli di AR Foundation, come `ARPlaneManager` e `ARTrackedImageManager`. Questi script sfruttano i metodi del ciclo di vita di Unity (`Awake()`, `OnEnable()`, `Update()`, `OnDisable()`), richiamati automaticamente dal motore, ad esempio per aggiornare a ogni fotogramma gli oggetti 3D in base al movimento dello smartphone.

3.2 Camera

In un progetto senza AR la Camera è l'occhio del giocatore nello spazio virtuale. In AR svolge invece un doppio compito:

- **Movimento**: la camera virtuale deve muoversi nella scena 3D esattamente come il telefono si muove nella stanza reale. Il componente `ARPoseDriver` aggiorna il Transform della Main Camera con la posa (posizione e rotazione) stimata dallo SLAM (*Simultaneous Localization and Mapping*), ovvero la stima simultanea della posizione del dispositivo e della mappa dell'ambiente circostante.

- **Sfondo**: normalmente lo sfondo è un colore pieno o un cielo (*Skybox*). Tramite `ARCameraBackground`, AR Foundation lo sostituisce con il flusso video in tempo reale della fotocamera del telefono.

3.3 Sistema di coordinate

Unity usa uno spazio di coordinate cartesiane 3D con origine in $(0, 0, 0)$, in cui per convenzione un'unità corrisponde a un metro. ARCore e ARKit creano invece un proprio sistema di coordinate (*session space*), la cui origine coincide con la posizione del dispositivo all'avvio della sessione. In Unity, quando un GameObject è posto dentro un altro (relazione padre-figlio), le coordinate del figlio sono relative al padre. AR Foundation sfrutta questo meccanismo con un oggetto padre della camera (`XR Origin`), che converte il session space nel sistema di coordinate del mondo di Unity.

3.4 Prefab e istanziazione

Un *Prefab* è un oggetto salvato come risorsa riutilizzabile, da cui Unity crea copie (istanze) a runtime con la funzione `Instantiate()`. Quando il telefono rileva un piano fisico (un tavolo, un pavimento), AR Foundation non disegna la geometria dal nulla: `ARPlaneManager` istanzia il prefab assegnato per ogni piano rilevato, aggiornandolo o rimuovendolo al variare del rilevamento. Lo stesso vale per ancore e altri *trackable*. Lo sviluppatore usa invece `Instantiate()` per posizionare nella scena i propri contenuti virtuali.

3.5 Raycasting

Per capire "dove tocca il dito sullo schermo nel mondo reale" si usa il *raycasting*, cioè il lancio di un raggio invisibile dalla camera attraverso un punto dello schermo, per trovare ciò che incontra lungo il suo percorso:

- **Physics.Raycast**: è il raycasting standard di Unity. Il raggio (ottenibile con `Camera.ScreenPointToRay`) è testato contro i collider degli oggetti virtuali e restituisce il primo oggetto colpito.

- **ARRaycastManager**: applica lo stesso concetto, ma non usa il motore fisico di Unity: il raggio è testato contro i piani e i punti caratteristici (*feature point*) rilevati dal sistema AR. Il metodo `Raycast` restituisce una lista di `ARRaycastHit` ordinata per distanza, ciascuno con una `pose` (posizione e rotazione del punto colpito); il tipo di superfici considerate si sceglie con `TrackableType`.

La posa restituita da `ARRaycastManager` viene tipicamente usata per istanziare un prefab sulla superficie toccata dall'utente.

Capitolo 4

AR Foundation

AR Foundation[9][10][18] è un package di Unity (il game engine su cui è sviluppata l'applicazione; un *package* è un insieme di codice e risorse che si aggiunge a un progetto Unity) che espone un'unica API (Application Programming Interface, cioè l'insieme di classi e funzioni che lo sviluppatore può richiamare) per le funzionalità di realtà aumentata, come il tracciamento della posizione della camera, il rilevamento dei piani, i raycast e le ancore. È quindi un framework *cross-platform*, ovvero un insieme di strumenti e librerie che permette di scrivere il codice una sola volta e ottenere app per più sistemi operativi, in questo caso Android e iOS (oltre ad alcuni visori AR/XR), senza modifiche al codice. AR Foundation, però, definisce soltanto *che cosa* si può chiedere al sistema e non contiene l'implementazione di *come* farlo. Per questo si comporta da regista: Unity organizza le funzionalità AR in *subsystem* XR, interfacce generiche come quella dei piani, della camera o dei raycast, e il compito di realizzarle concretamente spetta ai **plug-in provider**, package separati da installare e abilitare per ciascuna piattaforma di destinazione: **ARCore XR Plugin** per Android e **ARKit XR Plugin** per iOS. Ogni plug-in implementa i subsystem di Unity e fa da traduttore, convertendo le richieste generiche di AR Foundation nelle chiamate dello SDK nativo della piattaforma. Uno *SDK* (Software Development Kit) è un insieme di librerie, strumenti e documentazione fornito dal produttore di una tecnologia: su Android è l'**ARCore SDK** di Google, mentre su iOS è il framework **ARKit** di Apple. Il plug-in appartiene quindi a Unity, lo SDK a Google o Apple, e lo sviluppatore scrive codice solo con l'API di AR Foundation. Le funzioni specifiche di ARCore, come Geospatial e Cloud Anchors, richiedono invece un package a parte, **ARCore Extensions**.

Il plug-in di ARCore usa la versione dello SDK che espone un'API in C (*ARCore SDK for Android NDK*, libreria `libarcore_sdk_c.so`) ed è per questo che entra in gioco l'**NDK**[14] (Native Development Kit), l'insieme di strumenti con cui Google permette di compilare per Android codice nativo, cioè codice C/C++ tradotto direttamente in istruzioni eseguibili dal processore del telefono. Il codice Java o Kotlin, invece, viene compilato in *bytecode DEX* (un formato intermedio indipendente dal processore) ed eseguito da ART (Android Runtime), la macchina virtuale

di Android; il codice nativo, non passando da questo livello, è in genere più veloce e permette di riutilizzare librerie C/C++ già esistenti, per cui viene preferito per le parti più critiche come la visione artificiale. Il risultato della compilazione nativa sono le *shared library*, file con estensione `.so` (shared object) che il sistema carica in memoria solo a runtime, cioè quando l'app è in esecuzione. Poiché ogni architettura di processore (ABI, ad esempio `arm64-v8a`) richiede la propria versione compilata, le librerie vengono collocate in cartelle separate per ABI (Application Binary Interface) ovvero l'interfaccia che definisce quali sono le istruzioni in linguaggio macchina da usare per fare le chiamate (system call) al kernel, il modo in cui devono essere passati i parametri per tali chiamate e come ottenere i valori di ritorno. Lo stesso meccanismo riguarda Unity stesso: con lo scripting backend ahead-of-time (AOT, prima dell'esecuzione, in fase di build; il contrario è JIT, Just In Time) **IL2CPP** (Intermediate Language To C++) il C# non diventa C++ direttamente, ma segue la catena C# → IL → C++ → codice macchina. Prima il compilatore C# (Roslyn) trasforma tutti gli script, compresi quelli dei package come AR Foundation e dei provider, in assembly `.dll` (Dynamic-Link Library) contenenti *IL* (Intermediate Language, il bytecode di .NET); poi IL2CPP converte l'IL in C++, e infine l'NDK compila questo C++ in codice macchina, dentro una libreria `.so`. Le librerie del plug-in e dello SDK ARCore, invece, sono distribuite già compilate, per cui nella build l'NDK serve soprattutto a IL2CPP. Durante la fase di *build*, la compilazione e il confezionamento del progetto, tutte queste librerie (quelle di Unity, quelle del plug-in e quelle dello SDK ARCore) vengono inserite nell'**APK** (Android Package Kit), il file di installazione di un'app Android, un archivio che contiene codice, risorse e manifest (per la pubblicazione su Google Play si usa invece l'**AAB**, Android App Bundle, da cui lo store ricava gli APK). Su iOS non esistono né NDK né file `.so`: il codice nativo viene compilato con Xcode in librerie statiche o framework.

Una volta installata l'app, il funzionamento a runtime segue una catena di chiamate che scende di livello a ogni passaggio. Quando uno script C# invoca, ad esempio, `ARRaycastManager.Raycast` per capire quale superficie reale si trova sotto il punto toccato sullo schermo, AR Foundation inoltra la richiesta al subsystem corrispondente, che a sua volta la passa al provider di ARCore XR Plugin. Fin qui sono tutte classi C# gestite, che con IL2CPP diventano semplici chiamate di funzione C++ all'interno dello stesso binario, senza alcun *P/Invoke*. Il *P/Invoke* (Platform Invoke), il meccanismo di .NET per chiamare funzioni native, serve infatti solo al confine tra il C# e il codice compilato a parte: il provider lo usa per raggiungere `libUnityARCore.so`, la libreria nativa di Unity già compilata inclusa nel plug-in, che il C# non può chiamare in altro modo. Da qui in giù non c'è più P/Invoke, ma normali chiamate C/C++ tra librerie condivise: `libUnityARCore.so` richiama l'API C dello SDK ARCore (`libarcore_sdk_c.so`). I dati restituiti dal lato nativo, come il buffer degli hit del raycast, risiedono in memoria nativa, che il Garbage Collector di .NET non gestisce: va quindi liberata esplicitamente dal codice C#, tramite una chiamata di rilascio alla libreria che l'ha allocata, altrimenti si genera un *memory leak*. Inoltre quel buffer appartiene al lato nativo e può essere

riutilizzato o invalidato al frame successivo, per cui il provider ne copia il contenuto in una `NativeArray` gestita dal chiamante e solo dopo rilascia la memoria nativa. La copia costa poco (gli hit sono pochi) e rende l'API sicura per chi la usa. Per le parti che invece vivono nel mondo Java, come la verifica dell'installazione di Google Play Services for AR o la richiesta dei permessi, si usa la **JNI** (Java Native Interface), il ponte che permette al codice nativo e a quello Java/Kotlin di chiamarsi a vicenda (in Unity spesso tramite `AndroidJavaObject`). Lo SDK, infine, non esegue da solo il tracciamento: comunica con il vero motore AR di sistema, **Google Play Services for AR**, un servizio separato dall'app che si installa e si aggiorna tramite Play Store, il quale analizza i frame della fotocamera e i dati dei sensori inerziali (accelerometro e giroscopio) per stimare la posizione del dispositivo e individuare i piani. Su iOS questo ruolo è svolto da ARKit, già integrato nel sistema operativo e quindi privo di installazione separata, dato che si aggiorna con iOS. I risultati compiono poi il percorso inverso: il motore AR restituisce i dati allo SDK, il plug-in li converte nelle strutture generiche dei subsystem e AR Foundation li espone allo sviluppatore, ad esempio sotto forma di oggetti `ARPlane`, aggiunti, aggiornati o rimossi man mano che il sistema rileva o affina i piani (evento `planesChanged`), permettendo di posizionare i contenuti virtuali sulla scena reale.

Una lista di tutte le funzionalità che AR Foundation supporta:

Caratteristica	**Descrizione**
Sesssion	Attivare, disattivare e configurare la realtà aumentata sulla piattaforma di destinazione.
Device tracking	Tracciare la posizione e la rotazione del dispositivo nello spazio fisico.
Camera	Esegui il rendering delle immagini provenienti dalle fotocamere del dispositivo ed effettua la stima della luce
Plane Detection	Rileva e traccia le superfici piane.
Bounding Box Detection	Rileva e traccia i riquadri di delimitazione degli oggetti 3D.
Image Tracking	Rileva e traccia immagini 2D.
Markers	Traccia i marcatori AR e leggi i dati codificati in essi.
Object Tracking	Rileva e traccia oggetti 3D.
Face Tracking	Rilevare e tracciare i volti umani.
Body Tracking	Individuare e tracciare un corpo umano.
Point Clouds	Individua e traccia i punti caratteristici.
Ray Casts	Proietta raggi contro gli oggetti tracciati.
Anchors	Tracciare punti arbitrari nello spazio.
Meshing	Genera le mesh dell'ambiente.
Enviroment Probes	Genera mappe cubiche dell'ambiente.
Occlusion	Occulta i contenuti AR con oggetti fisici ed esegui la segmentazione umana.
Participants	Traccia altri dispositivi in una sessione AR condivisa.

Non tutte le funzionalità sono supportate da tutte le piattaforme(Plug-in di Unity):

Caratteristica	Android	iOS	XR Sim.
Sesssion	Si	Si	Si
Device tracking	Si	Si	Si
Camera	Si	Si	Si
Plane Detection	Si	Si	Si
Bounding Box Detection		Si	Si
Image Tracking	Si	Si	Si
Markers			
Object Tracking		Si	
Face Tracking	Si	Si	
Body Tracking		Si	
Point Clouds	Si	Si	Si
Ray Casts	Si	Si	Si
Anchors	Si	Si	Si
Meshing		Si	Si
Enviroment Probes	Si	Si	Si
Occlusion	Si	Si	Si
Participants		Si	

4.0.1 Architettura

L'architettura di Unity AR Foundation [9] si basa su tre concetti fondamentali, pensati per astrarre le differenze tra gli SDK nativi di realtà aumentata: **Subsystems**, **Managers** ed **Extensions**.

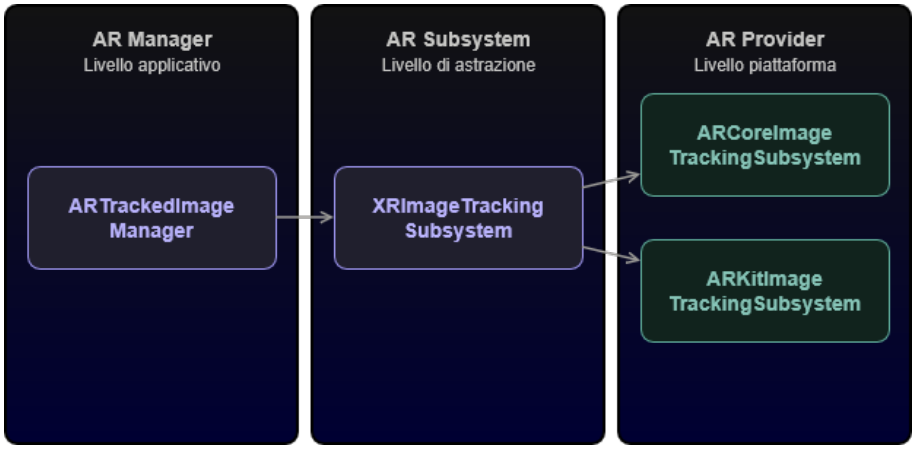


Figura 4.1: AR Foundation Workflow (Example Feature: Image Tracking) [11]

La figura mostra come collaborano i diversi livelli per una funzionalità di esempio, il tracciamento di immagini. Al livello più alto si trova `ARTrackedImageManager`, un componente (`MonoBehaviour`) che lo sviluppatore aggiunge al GameObject `XR`

`Origin` della scena. Serve a gestire gli eventi e ad accedere alle immagini tracciate, e permette di assegnare una *Reference Image Library* (la lista di immagini da riconoscere) e un *Prefab* da istanziare sopra l'immagine quando viene rilevata. Al livello intermedio si trova `XRImageTrackingSubsystem`, il sottosistema che definisce l'interfaccia standard di Unity per il tracciamento di immagini, indipendente dal dispositivo. Stabilisce le regole e i metodi che ogni piattaforma (Android, iOS, ecc.) deve implementare, garantendo così il principio cross-platform. Al livello più basso si trova il provider specifico della piattaforma, ovvero l'implementazione concreta fornita dal plug-in: ARCore XR Plugin su Android, ARKit XR Plugin su iOS. Il provider traduce i comandi generici del sottosistema in chiamate allo SDK nativo, che comunica con gli algoritmi di visione artificiale di ARCore per riconoscere, attraverso la fotocamera, i punti caratteristici (*feature points*) dell'immagine di riferimento. Quando l'app viene compilata per Android, Unity attiva automaticamente il provider corrispondente.

Subsystems

Un **sottosistema**, implementato in Unity tramite la classe base `SubsystemWithProvider` [9], definisce il ciclo di vita e l'interfaccia di scripting di una specifica funzionalità del motore. Ad esempio, `XRPlaneSubsystem` definisce l'interfaccia di tracciamento dei piani: lo stesso codice applicativo può usare un piano rilevato sia su iOS sia su Android, perché AR Foundation non contiene le implementazioni a basso livello. Queste sono fornite dai **provider**, distribuiti in pacchetti detti **plug-in provider** (*Google ARCore XR* per Android, *Apple ARKit XR* per iOS), e costituiscono il livello di astrazione tra il codice applicativo e gli SDK proprietari.

Tutti i sottosistemi condividono lo stesso ciclo di vita (creazione, avvio, arresto, distruzione), gestito dall'`XRLoader` attivo nel progetto: ogni plug-in provider include il proprio loader, che istanzia i sottosistemi all'avvio dell'app e li distrugge alla chiusura (configurabile da *Project Settings* > *XR Plug-in Management*).

Ogni sottosistema ha un **descrittore** (`SubsystemDescriptor`) che indica le funzionalità supportate. Poiché queste dipendono da hardware e SDK, vanno verificate a runtime prima dell'uso:

```
var trackedImageManager = FindFirstObjectByType<ARTrackedImageManager>();

// Il provider supporta la modifica a runtime della libreria immagini?
if (trackedImageManager.descriptor.supportsMutableLibrary)
{
    // Logica specifica per librerie dinamiche
}
```

I **sottosistemi di tracciamento** rilevano e seguono entità fisiche dell'ambiente dette **trackable** (ad esempio `XRPlaneSubsystem` rileva i `BoundedPlane`). Ogni trackable ha un **TrackableId**, un GUID (Globally Unique Identifier) a 128 bit che lo identifica in modo univoco, durante la sessione AR, nelle fasi di aggiunta, aggiorna-

mento e rimozione. In `UnityEngine.XR.ARSubsystems` i trackable sono `struct` ad alte prestazioni che implementano `ITrackable`; in `UnityEngine.XR.ARFoundation` sono invece componenti `MonoBehaviour` (es. `ARPlane`) che le incapsulano.

Managers

Un **Manager** è un componente Unity (`MonoBehaviour`) che espone un'interfaccia ad alto livello per configurare una specifica funzionalità AR (piani, immagini tracciate, volti). I Manager ereditano da `SubsystemLifecycleManager` e fanno da ponte tra i dati dei sottosistemi XR e gli oggetti della scena, sincronizzandone il ciclo di vita con quello del componente: in `OnEnable` invocano `Start()` sul sottosistema associato, in `OnDisable` invocano `Stop()`.

I *Trackable Manager* (`ARPlaneManager`, `ARTrackedImageManager`, ...) creano, aggiornano e distruggono automaticamente i `GameObject` dei trackable, istanziandoli come figli di `XR Origin` e posizionandoli rispetto al suo spazio di riferimento. Si può assegnare un *Prefab* per la loro rappresentazione visiva (ad esempio una mesh per un piano); se manca, il Manager crea un `GameObject` vuoto con il componente `ARTrackable` idoneo, che aggiunge a runtime anche a un Prefab che ne fosse privo.

Extensions

AR Foundation non espone tramite astrazioni C# tutte le API dei singoli SDK nativi: offre ciò che ARKit, ARCore e gli altri provider hanno in comune, mentre le funzionalità specifiche di una piattaforma restano fuori. Per colmare questo divario fornisce puntatori nativi grezzi, la proprietà nativePtr (di tipo IntPtr), che permettono di accedere direttamente alle strutture del provider. È presente, ad esempio, su XRSessionSubsystem e sulla maggior parte dei trackable (piani, immagini, volti), e tramite P/Invoke o un plug-in nativo consente di recuperare oggetti interni della piattaforma come ARFrame, la sessione nativa (ARSession su iOS, ArSession su Android) e gli elementi geometrici dell'ambiente. Struttura e indirizzo sono definiti da ciascun plug-in provider, quindi il layout esatto va verificato nella sua documentazione e nei suoi header. Per convenzione il puntatore indirizza una struttura versionata il cui primo membro è un intero con la versione dell'interfaccia, seguito dai puntatori grezzi:

```
1 typedef struct UnityXRNativeSessionPtr
2 {
3     int version;   // offset 0 (4 byte), seguiti da 4 byte di padding su 64
      bit
4     void* session; // offset 4 su 32 bit, offset 8 su 64 bit
5 } UnityXRNativeSessionPtr;
```

Il campo `session` non si trova a 4 byte dall'inizio della struttura a causa dell'allineamento in memoria (*struct padding*): le CPU accedono più efficientemente ai dati il cui indirizzo è multiplo della loro dimensione, quindi il compilatore inserisce

byte inutilizzati per portare ogni membro a un offset adeguato. Su 32 bit `session` si trova a offset 4, su 64 bit a offset 8. Usare in C# un offset fisso di 4 byte restituirebbe quindi un valore errato sui dispositivi a 64 bit, cioè praticamente tutti quelli attuali, perché si leggerebbe il padding o metà del puntatore.

Il puntatore va usato con cautela: bisogna verificare che non sia `IntPtr.Zero` (provider che non lo supporta o oggetto non più valido) e leggere `version` per sapere quali campi sono disponibili. Non va mai liberato, perché appartiene al provider e può diventare non valido alla distruzione della sessione o alla rimozione del trackable, quindi non va conservato ma riletto quando serve. Inoltre il tipo dipende dalla piattaforma: su iOS `ARSession` e `ARFrame` sono oggetti Objective-C e richiedono codice nativo (Objective-C/Swift), mentre su Android i puntatori sono tipi dell'NDK di ARCore (`ArSession*`, `ArFrame*`), da usare con le API C e non con gli oggetti Java/Kotlin. Poiché `nativePtr` lega il codice a piattaforma e provider, conviene isolarlo con la compilazione condizionale (`#if UNITY_IOS`, `#if UNITY_ANDROID`) o dietro un'interfaccia propria.

4.0.2 Componenti Principali

AR Foundation[11] prevede l'uso di alcuni concetti fondamentali, i quali sono comuni a tutte le AR App.

AR Session

Il GameObject AR Session gestisce lo stato del dispositivo AR e la sessione corrente: quando tutti i componenti sono inizializzati e funzionano correttamente, lo stato assume il valore di *SessionTracking*. Questo componente controlla l'intero ciclo di vita dell'esperienza di realtà aumentata. Contiene inoltre lo script AR Input Manager, fondamentale per abilitare il tracciamento dello spazio circostante.

XR Origin

Il GameObject XR Origin converte i dati di tracciamento del dispositivo (come punti di interesse o piani detected) nello spazio coordinate di Unity. Ciò garantisce che la scena virtuale sia perfettamente sincronizzata con il mondo reale e ospita al suo interno la AR Camera. Mentre gli elementi tracciabili vengono gestiti dal dispositivo nello "spazio sessione" (relativo al sistema di coordinate locale del dispositivo), i GameObject in Unity vengono posizionati nello "spazio mondo".

Tracked Pose Driver (InputSystem)

Il Tracked Pose Driver è un componente applicato alla fotocamera AR che ne aggiorna la posizione e la rotazione locali in tempo reale, basandosi sulle informazioni di tracciamento fornite dal sensore/dispositivo.

AR Camera Manager

L'AR Camera Manager gestisce il flusso video della fotocamera principale del dispositivo, combinando lo sfondo reale con gli elementi grafici 3D virtuali. Attraverso questo script è possibile configurare l'autofocus (di norma consigliato attivo, tranne nei contesti di inquadratura esclusivamente a lunga distanza). Consente inoltre di attivare la stima della luminosità (light estimation) per aggiornare la sorgente di luce della scena Unity e di effettuare lo switch tra la fotocamera posteriore e quella frontale.

4.0.3 Perché AR Foundation?

Uno **SDK** (Software Development Kit)[13] è un insieme di strumenti forniti dal produttore di una piattaforma per sviluppare software per essa: compilatore, librerie con un'interfaccia pubblica detta **API** (Application Programming Interface), che permette a software diversi di comunicare, documentazione e informazioni sulle licenze di distribuzione. Gli SDK sono quindi **specifici per un ecosistema**: nel caso della realtà aumentata, ARKit di Apple funziona sostanzialmente solo su iOS e ARCore di Google solo su Android, e **il codice scritto per uno non è riutilizzabile sull'altro**.

Sviluppare applicazioni AR mobile con gli SDK nativi (Java/Kotlin o C/C++ tramite NDK) comporta quindi costi di produzione più alti, scarsa portabilità del codice e una gestione degli aggiornamenti separata per ogni piattaforma. Unity usa l'API C di ARCore, perché lavora in C/C++ e C# e può così chiamarla evitando, per le funzioni principali, il passaggio dal livello Java/ART.

AR Foundation nasce proprio **per evitare di scrivere e mantenere due codici diversi per la stessa funzione**, introducendo un livello di astrazione che traduce le istruzioni verso i rispettivi SDK.

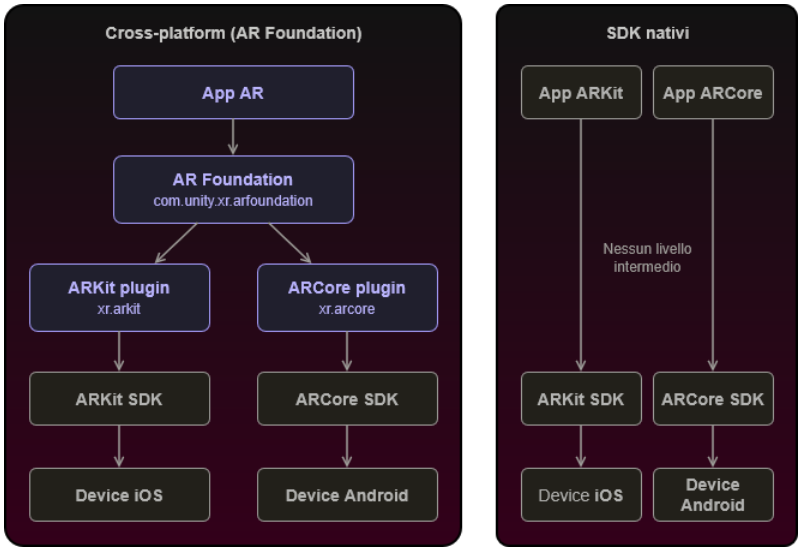


Figura 4.2: AR Foundation Architecture vs Native Architecture [24]

- **ARKit XR Plugin** (`com.unity.xr.arkit`) e **ARCore XR Plugin** (`com.unity.xr.arcore`): package Unity ufficiali, gestiti tramite lo Unity Package Manager (UPM). Agiscono da *provider* nell'architettura XR di Unity: contengono codice C# e librerie native (`.a` per iOS, `.so` per Android) che fanno da ponte tra l'interfaccia astratta di AR Foundation e l'SDK nativo.

- **ARKit e ARCore (SDK nativi)**: non sono package Unity, ma i framework forniti da Apple e Google. Eseguono il tracciamento e la visione artificiale a basso livello: ARKit è integrato nel sistema operativo, mentre ARCore si appoggia al servizio di sistema Google Play Services for AR.

- **Accesso diretto alle funzioni native**: AR Foundation non funziona da sola, ma sempre tramite i provider. Per funzioni esclusive non ancora astratte, lo sviluppatore può usare le API esposte dai plug-in (ad esempio `ARKitSessionSubsystem` o `ARCoreSessionSubsystem`) senza scrivere plug-in C/C++ personalizzati. Per ARCore, funzioni come Geospatial e Cloud Anchors richiedono il package aggiuntivo **ARCore Extensions**.

Il vantaggio principale di AR Foundation è lo sviluppo con un'unica base di codice (*single codebase*) valida sia per iOS sia per Android, che riduce tempi di sviluppo, manutenzione e costi di produzione. A ciò si aggiungono la rapidità di prototipazione, grazie alla configurazione dei componenti direttamente nell'editor di Unity, e l'integrazione nativa con l'ecosistema Unity: pipeline di rendering, profiler, sistema UI e Visual Scripting.

Gli svantaggi sono due. Il primo è che le novità hardware e le nuove API di Google o Apple non sono disponibili subito: bisogna attendere che Unity le supporti, e un ritardo in un aggiornamento o in un bugfix può bloccare il progetto. Il secondo è che il livello di astrazione introduce un overhead, con un impatto (generalmente contenuto) su framerate e uso di CPU e RAM.

L'approccio nativo con ARCore, privo di livelli intermedi, offre invece le massime prestazioni, un consumo di risorse minore, accesso immediato alle ultime funzionalità e integrazione profonda con i servizi della piattaforma, oltre all'accesso di basso livello a sensori, fotocamera e memoria. Il prezzo è però elevato: una curva di apprendimento più ripida, codice più verboso (*boilerplate*), competenze specifiche per ciascuna piattaforma e una doppia manutenzione. Un'app con ARCore nativo funziona solo su Android, e per coprire iOS bisogna riscrivere la logica con ARKit, mantenendo due basi di codice da aggiornare e testare nel tempo.

Capitolo 5

App

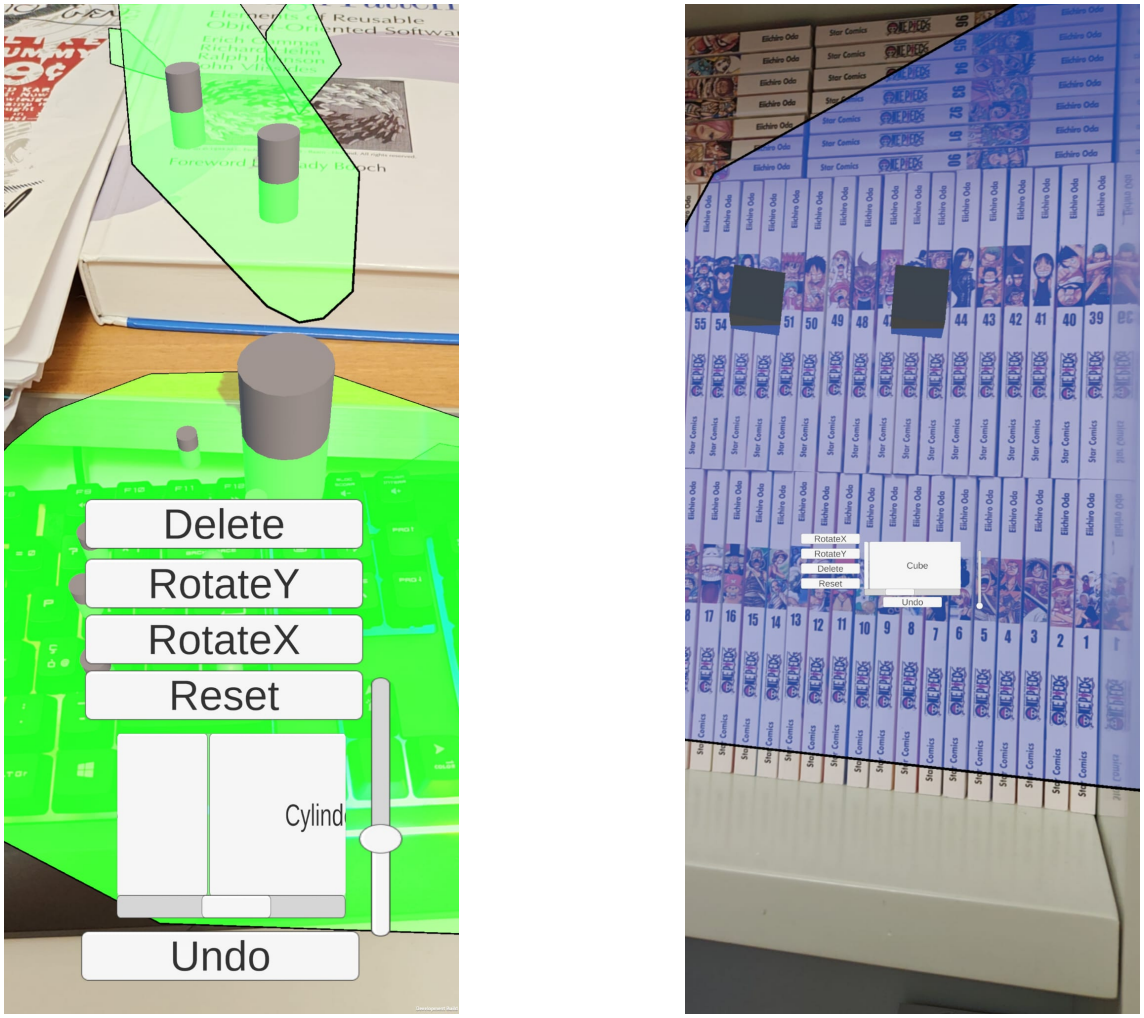


Figura 5.1: Screenshots while my app is running on my phone

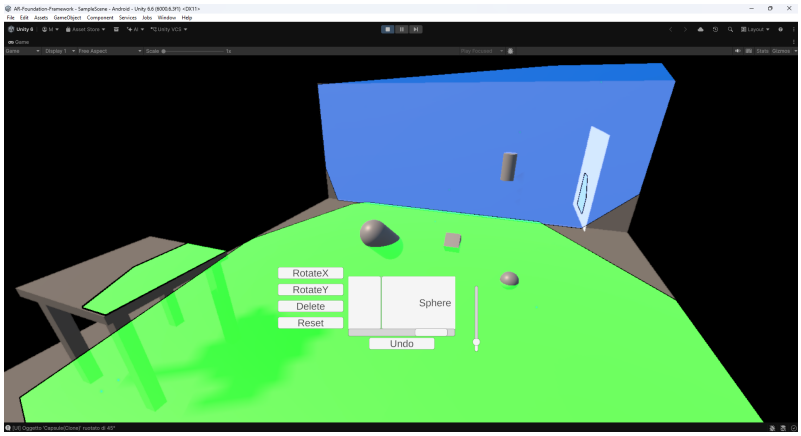


Figura 5.2: Screen while my app is running in Unity XR Simulator

5.1 Analisi Dei Requisiti

5.1.1 Scopo e Uso

L'applicazione sviluppata in questo progetto è un app AR di tipo Markerless e Video See-Through, la quale consiste nella creazione dinamica di oggetti tridimensionali su superfici piane, con possibilità di eliminare, ruotare, scalare, spostare con drag and drop, pulire la scena e applicare `"l'undo"` alle azioni dell'utente (annullamento dell'azione), il quale interagisce tramite una semplice GUI composta da pulsanti, slider e per quanto riguarda `l'XR Simulator` con un input che prevede gestione del click del mouse, di una telecamera virtuale ARCamera e la possibilità di muoversi con WASD con la tastiera nell'editor di Unity e invece per quanto riguarda la distribuzione mobile del touch del touchscreen e della telecamera e sensori del cellulare.

Quest'app è stata distribuita su Android usando AR Foundation con ARCore XR Plug-In. Come si può presumere, l'app distribuita su smartphone avrà un margine d'errore maggiore rispetto all'editor, derivato dal fatto che non si hanno superfici e telecamere/sensori ideali e stabili come nell'editor, infatti ciò può portare ad ulteriori ritardi, errori, saturazione e ha tanti altri fattori negativi.

Per la realizzazione di tale software, ho fatto uso di 3 manager di AR principali, ovvero `ARPlane, ARRaycast e ARPointCloud` aggiunti come componenti all'`XR Origin`, con in aggiunta due manager custom creati da me per personalizzare `ARPlaneManager e ARPointCloudManager`, facendo in modo, rispettivamente, di vedere tre colori diversi per superfici verticali e orizzontali e quando l'oggetto è in stato di `"drag"` e stessa cosa per quanto riguarda i `"point cloud"`, un colore di default e l'altro quando il drag è attivo. Esempio di parte di codice usato per CustomPlaneManger:

```
1 public class CustomPlaneManager : MonoBehaviour
2 {
3     //...
```

```
4       private void OnPlanesChanged(ARTrackablesChangedEventArgs<ARPlane>
    args)
5       {
6           foreach (var plane in args.added)
7               ApplyDefaultPlaneColor(plane);
8           foreach (var plane in args.updated)
9           {
10              if (plane != currentlyActivePlane)
11                  ApplyDefaultPlaneColor(plane);
12          }
13      }
14      private void ApplyDefaultPlaneColor(ARPlane plane)
15      {
16          if (plane == null) return;
17          Color targetColor = plane.alignment switch
18          {
19              PlaneAlignment.HorizontalUp or PlaneAlignment.HorizontalDown =>
     horizontalColor,
20              PlaneAlignment.Vertical => verticalColor,
21              _ => defaultColor //_ scelta default
22          };
23          SetPlaneColor(plane, targetColor);
24      }
25      private void SetPlaneColor(ARPlane plane, Color color)
26      {
27          Renderer renderer = plane.GetComponent<Renderer>() ?? plane.
    GetComponentInChildren<Renderer>();
28          if (renderer != null && renderer.material != null)
29          {
30              renderer.material.color = color;
31          }
32      }//...
33  }
```

In particolare, i 3 componenti AR usati sono:

- **ARPointCloudManager** serve a rilevare i "feature points" (punti di interesse), ovvero elementi visivi distintivi (angoli, trame, contrasti) che la fotocamera del dispositivo usa per orientarsi nello spazio; genera una "nuvola di punti" tridimensionale che rappresenta la struttura grezza dell'ambiente circostante e viene utilizzato principalmente dal sistema interno di tracciamento (SLAM) per capire come si muove il dispositivo ed è utile durante lo sviluppo o nelle fasi di calibrazione per mostrare all'utente che il sistema sta effettivamente "vedendo" l'ambiente.

- **ARPlaneManager** analizza la nuvola di punti e le immagini della fotocamera per identificare e ricostruire superfici piane reali; riconosce e traccia superfici piatte, classificandole automaticamente in piani orizzontali (es. pavimenti, tavoli), piani verticali (es. pareti, porte) e piani inclinati ed unisce automaticamente piani adiacenti se rileva che fanno parte della stessa superficie e aggiorna i loro confini in tempo reale man mano che l'utente inquadra l'ambiente.

- **ARRaycastManager** permette di generare dei "raggi" virtuali dallo schermo del dispositivo verso l'ambiente reale per trovare punti di intersezione con gli elementi rilevati; esegue un test di collisione (raycasting) non contro i normali

collider di Unity, ma contro i dati AR generati dall'ambiente (superfici tracciate dall'ARPlaneManager o feature points dell'ARPointCloudManager), per questo per il sistema di `drag and drop` degli oggetti ho fatto uso del raycast fisico che si basa sui collider; è essenziale per l'interazione dell'utente, poichè quando quest'ultimo tocca lo schermo, l'ARRaycastManager determina l'esatto punto nello spazio 3D reale in cui posizionare un oggetto.

5.1.2 Requisiti Funzionali

RF1 - Interfaccia Grafica

- **RF1.1** - Il sistema deve mostrare i pulsanti che agiscono sull'oggetto, ovvero Delete, CleanAllObject, Rotate(x and y) and Spawn
- **RF1.2** - Il sistema deve mostrare il pulsante per l'annullamento delle azioni
- **RF1.3** - Il sistema deve mostrare una Scroll View per selezionare l'oggetto da spawnare
- **RF1.4** - Il sistema deve mostrare uno Slider per cambiare la scala di un oggetto selezionato a runtime

RF2 - Sistema Di Input

- **RF2.1** - Il sistema deve avere accesso alla telecamera del dispositivo mobile oppure una camera virtuale di un simulatore
- **RF2.2** - Il sistema deve avere accesso a vari sensori del dispositivo
- **RF2.3** - Il sistema deve consentire all'utente di interagire tramite il touch da mobile (il movimento con WASD qua è camminare nel mondo reale) e con il mouse e tastiera(WASD) dal simulatore.

RF3 - Sistema Di Output e Comportamento

- **RF3.1** - Il sistema deve poter intercettare e visualizzare a schermo le superfici piane(verticali, orizzontali o inclinati), in corrispondenza con il mondo reale
- **RF3.2** - Il sistema deve poter intercettare e visualizzare a schermo le varie point clouds, in corrispondenza con il mondo reale
- **RF3.3** - Il sistema deve poter interpretare e capire quando vede un piano tramite operazione di Raycast

5.1.3 Requisiti Non Funzionali

RNF1 – Usabilità

- **RNF1.1** - I comandi devono essere intuitivi e facilmente accessibili (`WASD`/frecce, mouse, pulsanti, scroll view, slider).
- RNF1.2 - L'interfaccia deve essere chiara e mostrare informazioni essenziali (comandi e output).
- RNF1.3 - Il sistema deve essere comprensibile senza necessità di un tutorial complesso.

RNF2 – Affidabilità

Il sistema non deve andare in crash durante una sessione di uso standard.

RNF3 – Manutenibilità

- RNF3.1 - Il codice deve essere strutturato in moduli separati.
- RNF3.2 - Il sistema deve permettere l'aggiunta di nuovi prefab o nuove funzioni senza modificare pesantemente il codice esistente.

RNF4 – Portabilità

- RNF4.1 - Il sistema deve essere eseguibile sui sistemi operativi scelti.
- RNF4.2 - Il sistema non deve dipendere da librerie non portabili o non supportate.

RNF5 – Sicurezza

Il sistema deve impedire input non validi o fuori dai limiti

RNF6 – Qualità grafica

Gli asset grafici devono essere leggibili e non confondere l'utente.

RNF7 – Scalabilità

Il sistema deve poter gestire l'aggiunta di ulteriori elementi senza degradare significativamente le prestazioni.

5.1.4 UML Diagrams

Ho diviso in più parti i diagrammi Activity e Class per renderli più visibili.

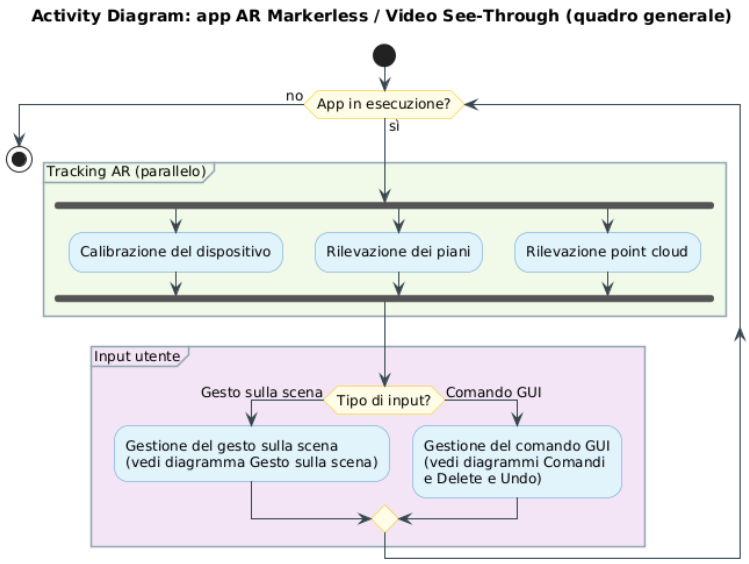


Figura 5.3: Activity Diagram, Focus: Generale

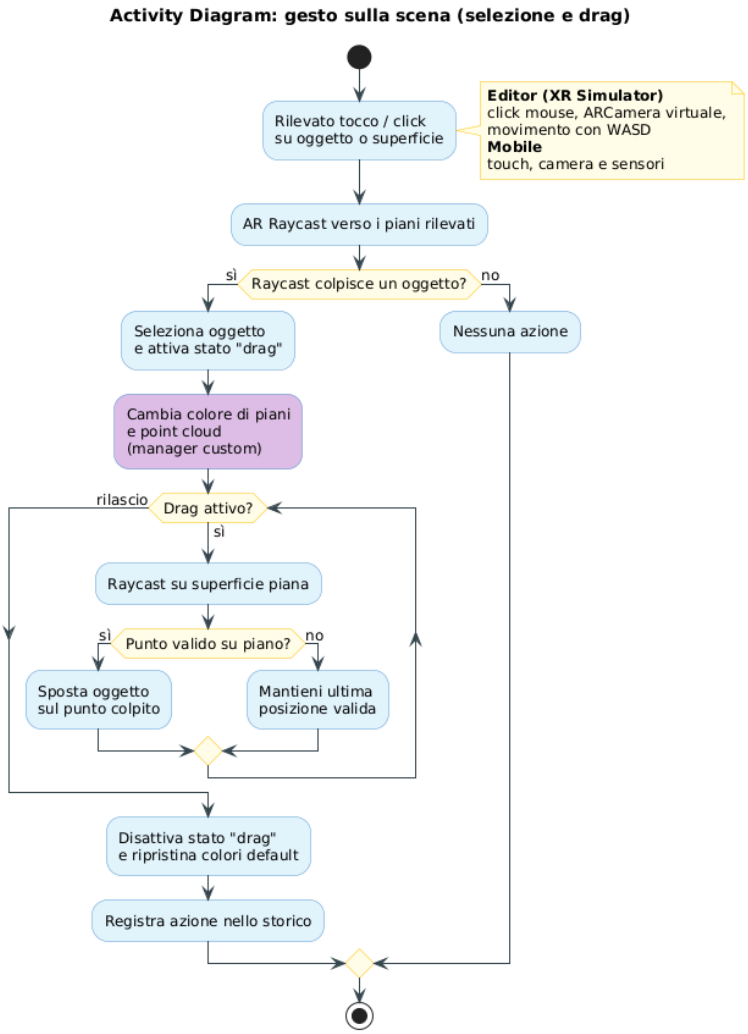


Figura 5.4: Activity Diagram, Focus: Selezione e Drag

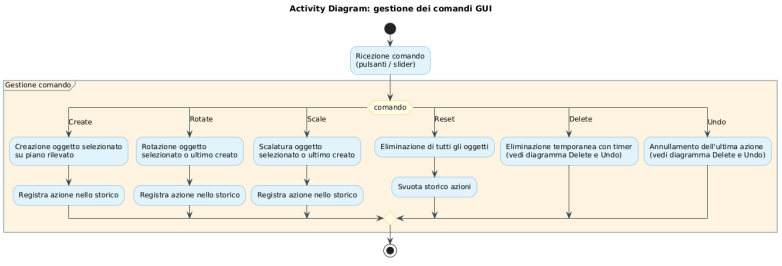


Figura 5.5: Activity Diagram, Focus: GUI

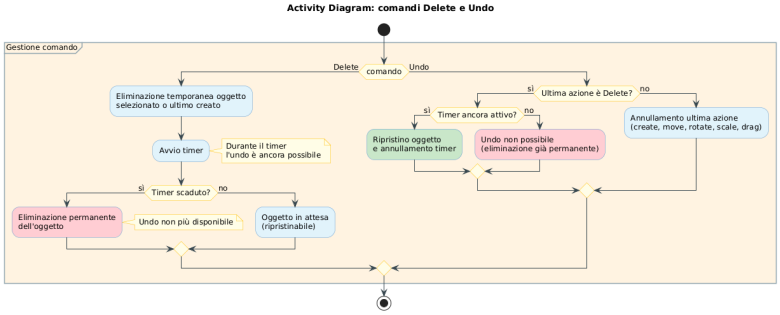


Figura 5.6: Activity Diagram, Focus: GUI Delete e Undo

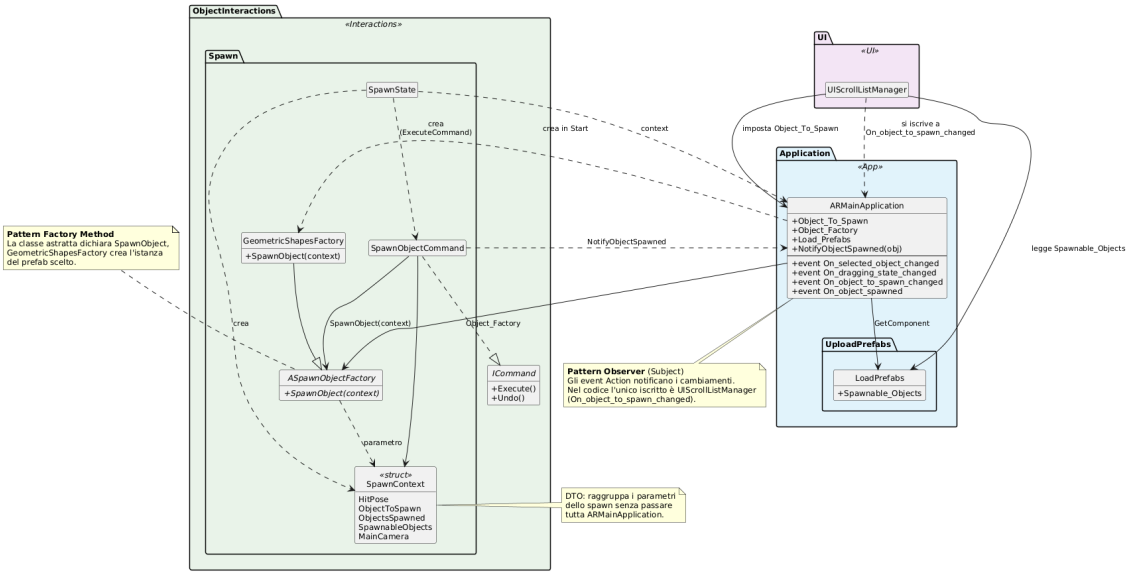


Figura 5.7: Class Diagram, Focus: Factory e Observer

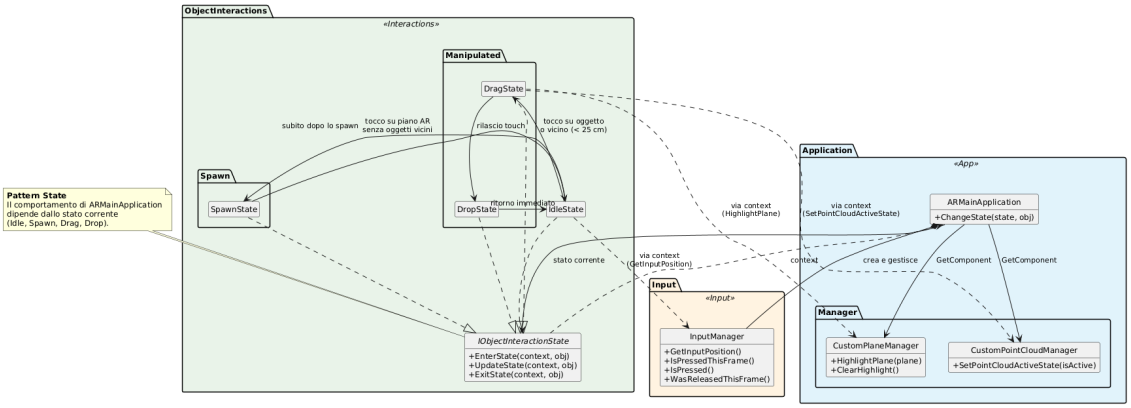


Figura 5.8: Class Diagram, Focus: State

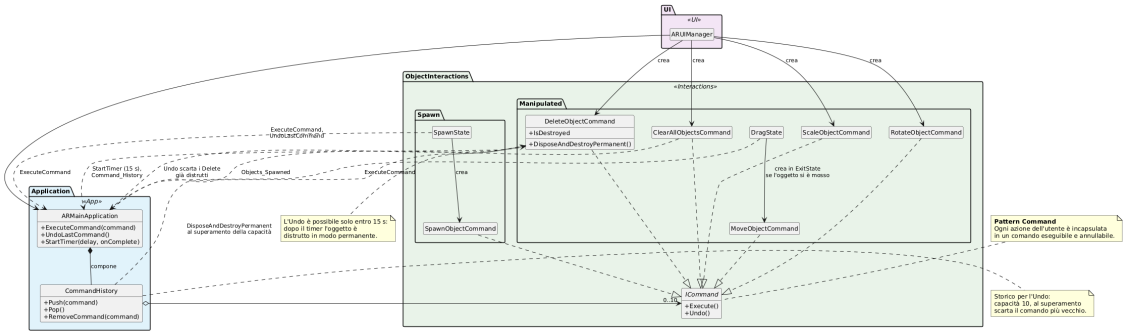


Figura 5.9: Class Diagram, Focus: Command

5.2 Design Pattern

5.2.1 Convenzioni di Nomenclatura

- Le **classi astratte** sono prefissate con `A`.
- Le **interfacce** sono prefissate con `I`.
- I **metodi** e le **classi concrete** seguono *QuestoCase*.
- Le **variabili** sono tutte minuscole con separatore `_`.
- Le **variabili con Setter e Getter** sono tutte con l'iniziale maiuscola con separatore `_`.
- le variabili evento observer Action (**eventi observer Action**) dell'engine sono con una sola iniziale maiuscola (obbligatorio) e il resto in minuscolo con separatore `_`.

5.2.2 Command

Design pattern comportamentale e consente di incapsulare un'azione dentro ad un oggetto, diasaccopiando chi invoca l'azione da chi la esegue. Nel progetto è stato usato per le azioni che può compiere l'utente, integrando anche il meccanismo di annullamento dell'azione (undo). (Vedi in Factory Method per l'implementazione di un comando)

```
1 public interface ICommand
2 {
3     void Execute();
4     void Undo();
5 }
```

5.2.3 State

Design pattern comportamentale e consente ad un oggetto di cambiare il proprio comportamento a runtime in base allo stato in cui si trova e quindi di cambiare anche quest'ultimo. Nel progetto è stato usato per rappresentare i vari stati diversi dell'interazione con gli oggetti.

```
public interface IObjectInteractionState
{
    void EnterState(ARMainApplication context, GameObject obj);
    void UpdateState(ARMainApplication context, GameObject obj);
    void ExitState(ARMainApplication context, GameObject obj);
}

public class SpawnState : IObjectInteractionState
{
    public void EnterState(ARMainApplication context, GameObject obj)
    {
        if (context.Object_Factory != null && context.Hit_List.Count > 0)
        {
            Pose hit_pose = context.Hit_List[0].pose;
            SpawnContext spawn_context = new SpawnContext(
                hit_pose,
                context.Object_To_Spawn,
                context.Objects_Spawned,
                context.Load_Prefabs.Spawnable_Objects,
                context.Main_Camera
            );

            // Invece di chiamare direttamente lo Spawn, incapsuliamo nel Command
            ICommand spawn_cmd = new SpawnObjectCommand(context.Object_Factory, spawn_context, context);
            context.ExecuteCommand(spawn_cmd);
        }
        context.ChangeState(new IdleState(), null);
    }//...
}
public class ARMainApplication : MonoBehaviour
{
    //...
    public void ChangeState(IObjectInteractionState newState, GameObject targetObj)
    {
        current_state?.ExitState(this, targetObj);
        current_state = newState;
        current_state?.EnterState(this, targetObj);
    }
    //...
}
```

5.2.4 Observer

Design pattern comportamentale che definisce come gli oggetti comunicano quando uno cambia stato, notificando automaticamente tutti gli osservatori (Per Invoke guarda Factory Method). Nel progetto è stato usato per condividere informazioni

sugli stati dell'applicazione, come la modalità di trascinamento (dragging) o lo spawn di un nuovo oggetto, senza che le classi coinvolte si conoscano direttamente.

L'implementazione sfrutta i delegate di C#: ARMainApplication espone eventi di tipo Action (ad esempio On_dragging_state_changed), a cui gli altri componenti si iscrivono con l'operatore +=. La parola chiave event impedisce agli iscritti di invocare l'evento o di sovrascrivere gli altri osservatori, mantenendo l'incapsulamento. Quando lo stato cambia, il setter della proprietà richiama Invoke tramite l'operatore ?., che evita errori se nessun osservatore è iscritto. La notifica avviene solo se il valore è effettivamente cambiato, evitando chiamate inutili. In questo modo ARMainApplication (il subject) rimane disaccoppiato dagli osservatori, che possono essere aggiunti o rimossi senza modificarlo.

```
public class ARMainApplication : MonoBehaviour
{
    public event Action<bool> On_dragging_state_changed;
    //...
     public bool Is_Dragging
     {
         get => is_dragging;
         set
         {
             if (is_dragging != value)
             {
                 is_dragging = value;
                 On_dragging_state_changed?.Invoke(is_dragging); //
    Notifica Observer
             }
         }
     }
     //...
    public void NotifyObjectSpawned(GameObject spawnedObj)
   {
       On_object_spawned?.Invoke(spawnedObj);
   }

}
```

5.2.5 Factory Method

Design Pattern creazionale che consente di creare oggetti senza specificare esplicitamente la loro classe concreta, delegando la creazione alle sottoclassi. Nel progetto è stato usato per la creazione di oggetti.

```
public abstract class ASpawnObjectFactory
{
    public abstract GameObject SpawnObject(SpawnContext context);
}

public class SpawnObjectCommand : ICommand
{//...
    public void Execute()
    {
        spawned_object = factory.SpawnObject(spawn_context);
```

```
        if (spawned_object != null && ar_app != null)
        {
            ar_app.NotifyObjectSpawned(spawned_object);
        }
    }

    public void Undo()
    {
        if (spawned_object != null)
        {
            if (ar_app != null && ar_app.Objects_Spawned.Contains(
    spawned_object))
            {
                ar_app.Objects_Spawned.Remove(spawned_object);
            }
            Object.Destroy(spawned_object);
        }
    }
}
```

5.2.6 Struttura Dati: CommandHistory

è una struttura dati, creata da me, formata da una `LinkedList` di `ICommand`, implementato come uno stack, in modo da creare il meccanismo di `undo` ed eliminare l'undo o dei comandi per oggetti selezionabili che possono non coincidere con gli ultimi inseriti.

```
public class CommandHistory
{//...
    public void Push(ICommand command)
    {
        history.AddLast(command);
        // Se superiamo la capacita' massima, rimuoviamo il primo elemento
    (il piu' vecchio)
        if (history.Count > maxCapacity)
        {
            ICommand oldestCommand = history.First.Value;
            history.RemoveFirst();

            // Se il comando eliminato e' una cancellazione definitiva,
    puliamo la risorsa
            if (oldestCommand is DeleteObjectCommand deleteCmd)
            {
                deleteCmd.DisposeAndDestroyPermanent();
            }
        }
    }

    public ICommand Pop()
    {
        if (history.Count == 0) return null;

        ICommand lastCommand = history.Last.Value;
        history.RemoveLast();
        return lastCommand;
    }//...

```

```
29      //Serve in DeleteObjectCommand, poiche' l'oggetto selezionato puo'
        essere diverso dall'ultimo messo
30      public bool RemoveCommand(ICommand command)
31      {
32          if (command == null) return false;
33
34          LinkedListNode<ICommand> node = history.Find(command);
35          if (node != null)
36          {
37              history.Remove(node);
38              Debug.Log("[CommandHistory] Comando rimosso con successo dallo
        storico!");
39              return true;
40          }
41          Debug.LogWarning("[CommandHistory] Impossibile trovare il comando
        da rimuovere nello storico.");
42          return false;
43      }
44  }
```

5.3 Analisi delle Prestazioni e Profiling

Per la valutazione sperimentale e l'ottimizzazione dell'applicazione è stato impiegato il *Unity Profiler*[25] [18], strumento di diagnostica integrato nell'engine che monitora in tempo reale e con frequenza al frame (un *frame* è ciascuna immagine prodotta e mostrata a schermo) il carico computazionale sulla CPU, l'occupazione della memoria (RAM e *Managed Heap*) e l'efficienza della pipeline di rendering. Il suo scopo è isolare i colli di bottiglia (*bottlenecks*) e stabilire se i rallentamenti siano imputabili all'elaborazione logica, alle chiamate di disegno verso la GPU o alla gestione delle risorse di memoria.



Figura 5.10: Schermata generale dell'Unity Profiler durante l'esecuzione dell'applicazione sul dispositivo di test.

I test sono stati condotti *on-device* (direttamente sul dispositivo fisico e non nell'Editor di Unity), tramite build di debug collegata via ADB (*Android Debug*

Bridge, strumento di comunicazione tra computer e dispositivo Android) ad un dispositivo di fascia medio-alta, un **Motorola Edge 70** con:

- **Sistema Operativo:** Android 16.
- **SoC e CPU:** Qualcomm Snapdragon 7 Gen 4 (*SoC*, *System on Chip*: unico chip che integra CPU, GPU (1.15 GHz) e altri componenti), Octa-Core, con clock fino a 2.8 GHz.
- **GPU:** Adreno 722.
- **RAM:** 12 GB LPDDR5X (memoria RAM per dispositivi mobili, a basso consumo).
- **Sensori coinvolti nel tracciamento:** fotocamera, accelerometro, giroscopio e magnetometro, con un Sensor Hub dedicato (coprocessore a basso consumo che raccoglie i dati dei sensori) per la fusione dei dati di tracciamento *spatially-aware* (consapevole della posizione e dell'orientamento del dispositivo nello spazio).

5.3.1 Tipologie di Grafici e Metodologia di Lettura

Il profiler organizza la diagnostica in quattro tipologie di grafici, a diversi livelli di astrazione (dall'andamento temporale macroscopico al singolo microsecondo di esecuzione):

1. **Grafico ad Area Impilata (*Timeline Stacked Area Chart*):** presente nel modulo *CPU Usage*. Sull'asse x sono riportati i frame sequenziali, sull'asse y il tempo di esecuzione in ms. Le macro-categorie sono aree cromatiche sovrapposte: *Scripts* (codice C# eseguito, compreso quello dei package come AR Foundation, che nel frame analizzato ne costituisce la parte prevalente.), *Rendering* (preparazione dei comandi grafici da parte della CPU), *Garbage Collector* (meccanismo che libera automaticamente la memoria degli oggetti C# non più usati, sospendendo temporaneamente il codice), *Physics* (simulazione fisica), *VSync* (attesa della sincronizzazione con l'aggiornamento del display) e *UI* (interfaccia utente). Le linee orizzontali a 16 ms (60 FPS) e 33 ms (30 FPS) permettono di vedere il superamento della soglia del frame e l'impatto di ciascun sottosistema.
2. **Grafico a Linee (*Line Chart*):** usato nei moduli *Rendering* e *Memory*, mostra l'andamento nel tempo di conteggi e quantità (ad esempio batch, triangoli, memoria usata) frame per frame.
3. **Traccia di Esecuzione e Threading (*Flame Graph / Call Stack Timeline*):** nella sezione inferiore del profiler è una traccia a blocchi gerarchici annidati analoga a un diagramma di Gantt. Mostra due thread: il *Main Thread*

(thread principale, che esegue logica, script e preparazione dei comandi grafici) e il *Render Thread* (thread dedicato che traduce tali comandi in chiamate alla GPU). L'asse orizzontale misura la cronologia interna del singolo frame (in ms), mentre le righe mostrano le chiamate a funzione e l'esecuzione parallela sui thread, evidenziando stalli e sincronizzazioni (`WaitForGfxCommands`, `Semaphore.WaitForSignal`).

4. **Grafici a Barre e Distribuzione Statistica (*Breakdown Bar Charts e Highlights*):** usati in *Selection Highlights*, *Capture Highlights* e *Memory Breakdown*. Barre orizzontali proporzionali mostrano valore assoluto e percentuale delle risorse, con indicatori statistici (minimo, mediana, massimo) per valutare la stabilità della sessione. La mediana è il valore centrale: metà dei frame è più veloce e metà più lenta.

5.3.2 Testing

La cattura analizzata comprende 2000 frame. Il frame time minimo è 19.198 ms, quindi nessun frame rispetta il budget di 16.67 ms richiesto da un target di 60 FPS, e la mediana è 33.408 ms (circa 30 FPS). Il carico risulta più spesso limitato dalla CPU che dalla GPU (*CPU-bound*, cioè una condizione in cui la velocità di generazione dei frame è limitata dal tempo impiegato dal processore per eseguire il codice e preparare i dati), ma, come mostrano i dati aggregati (*Capture Highlights*), una parte rilevante del tempo di frame è attesa di sincronizzazione e non lavoro computazionale. Le analisi di dettaglio sono state svolte sul frame 808, scelto perché il suo tempo di CPU (32.782 ms) è prossimo alla mediana della cattura (33.408 ms) e quindi rappresentativo della condizione tipica.

Analisi del Frame Selezionato

CPU Usage e profilo temporale. Il modulo *CPU Usage* misura il tempo necessario alla CPU per completare le operazioni di ciascun frame. A fronte di un target di 60 FPS (budget $\leq$ 16.67 ms), l'andamento supera costantemente tale soglia, posizionandosi spesso attorno ai 33 ms (30 FPS). Tale valore può essere influenzato anche da un limite alla frequenza di aggiornamento (`Application.targetFrameRate`, VSync o cadenza del flusso video di ARCore, la piattaforma Google per la realtà aumentata su Android su cui poggia AR Foundation) e non dipendere solo dal sovraccarico della CPU. La traccia del *Main Thread* nel frame 808 è così ripartita:

Routine	Funzione	Durata
`Update.ScriptRun BehaviourUpdate`	Invoca il metodo `Update()` (eseguito una volta per frame) di tutti i `MonoBehaviour` attivi, cioè gli script Unity: punto d'ingresso del codice C# e core della logica. Include le routine della riga seguente.	18.33 ms
`ARSession.Update()`, `ARCameraManager. Update()`	Routine di AR Foundation (livello di Unity che unifica le piattaforme AR, qui basato su ARCore): sincronizzano e analizzano i dati di tracciamento (*feature points*, cioè punti caratteristici dell'immagine usati per stimare la posa del dispositivo, piani rilevati) e convertono i frame video della fotocamera.	fino a 17.07 ms
`PostLateUpdate. FinishFrameRendering`	Il *Main Thread* impacchetta i comandi grafici e li inoltra alla coda del *Render Thread*.	6.83 ms
`Gfx.WaitForGfx CommandsFrom MainThread`, `Semaphore.WaitFor Signal` (*Render Thread*)	Il *Render Thread* attende inattivo che il *Main Thread* termini il frame; il semaforo è un meccanismo di sincronizzazione a basso livello tra thread.	14.35 ms

Le durate sono *inclusive* e annidate: `ARSession.Update()` viene eseguita all'interno di `BehaviourUpdate`, quindi i valori non vanno sommati. Ne deriva che la quasi totalità del tempo speso negli `Update()` (17.07 su 18.33 ms) è dovuta ad AR Foundation e non agli script applicativi. Le attese del *Render Thread* confermano che esso resta in attesa del completamento dei calcoli sul *Main Thread*.

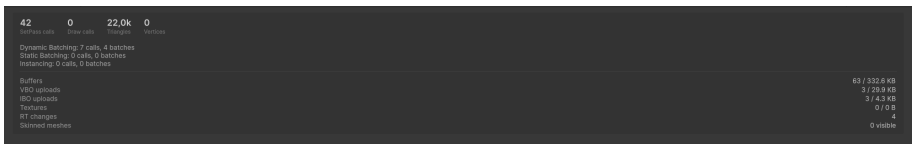


Figura 5.11: Scheda *Rendering* dell'Unity Profiler.

Rendering e complessità geometrica. Dai dati del modulo *Rendering* il carico grafico risulta contenuto e non rappresenta il principale collo di bottiglia. Il *Frame Debugger* (strumento che mostra frame per frame la sequenza dei comandi grafici inviati alla GPU) è raggiungibile dallo stesso modulo. I *SetPass Calls*, cioè i cambi di stato di rendering (shader, texture, blending) che comportano un overhead per

la CPU, sono 42, valore molto contenuto per un dispositivo mobile. Il batching dinamico (tecnica che unisce oggetti semplici in un unico gruppo da disegnare, detto *batch*) ha raggruppato 7 chiamate di disegno in 4 batch. La *Universal Render Pipeline* (URP) adotta inoltre l'*SRP Batcher*, meccanismo distinto dal batching dinamico che riduce il costo CPU dei cambi di materiale. La geometria conta 22.0k triangoli e 16.73k vertici, cifre che escludono problemi di saturazione della banda di memoria o di geometria eccessiva. Il *GPU Time* non è disponibile per il frame selezionato (*No Value*): la valutazione del carico grafico si basa quindi sulle statistiche di rendering e non su una misura diretta del tempo di GPU.

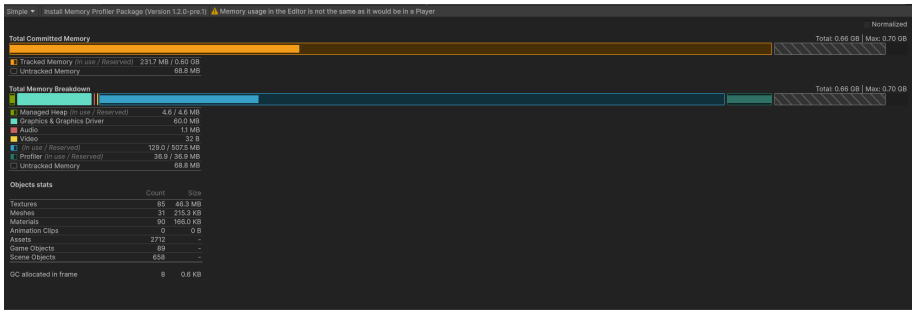


Figura 5.12: Scheda *Memory* dell'Unity Profiler.

Memory e Managed Heap. Il modulo distingue la memoria nativa di Unity (C++) dal *Managed Heap*, l'area di memoria in cui vengono allocati gli oggetti C# e la cui liberazione è affidata al *Garbage Collector* (GC) invece che al programmatore. I consumi globali sono ben controllati, ma critici nelle allocazioni temporanee:

- **Total Committed Memory:** 0.66 GB su un massimo di 0.70 GB. Indica la memoria complessivamente impegnata dall'applicazione, cioè richiesta al sistema operativo.
- **Tracked Memory:** 231.7 MB in uso su 0.60 GB riservati. È la memoria che Unity sa attribuire a oggetti e sistemi; *reserved* indica lo spazio già ottenuto dal sistema operativo, anche se non ancora in uso. A questa si aggiungono 68.8 MB di *Untracked Memory*, non attribuibili da Unity (ad esempio plugin nativi e driver).
- **Texture e Mesh:** 46.3 MB per 85 texture (immagini applicate alle superfici); 215.3 KB per 31 mesh (modelli geometrici 3D).
- **Managed Heap e Garbage Collection:** 4.6 MB in uso, ma con allocazioni continue ad ogni frame: nel frame selezionato 9 allocazioni per 0.7 KB (*GC Alloc*), originate principalmente da `Draw AR Background` (376 B) e dal *Render Graph* (128 B); sull'intera cattura le allocazioni sono 18.119, con un massimo di 43.9 KB in un singolo frame (frame 1396) e routine di AR Foundation tra

i principali contributori. Il valore istantaneo è ridotto, ma l'accumulo genera frequenti interventi bloccanti del GC, che sospende l'esecuzione del codice C# per ripulire la memoria non più utilizzata.

Analisi Statistica degli Highlights (Selection e Capture)

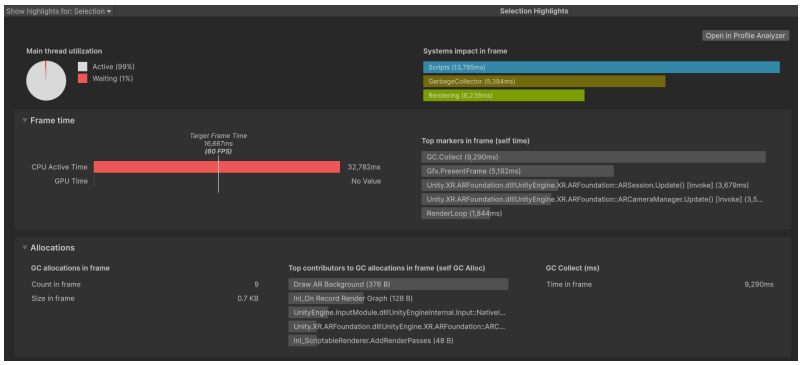


Figura 5.13: Scheda *Selection Highlights* dell'Unity Profiler.

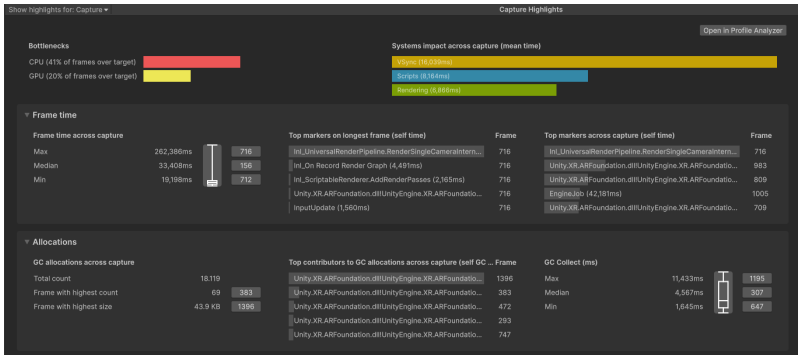


Figura 5.14: Scheda *Capture Highlights* dell'Unity Profiler.

Le schede *Selection Highlights* (SH, Figura 5.13), relative al singolo frame selezionato, e *Capture Highlights* (CH, Figura 5.14), che offrono una panoramica statistica dell'intera sessione, riportano le seguenti metriche:

- **Utilizzo del Main Thread (SH):** attivo per il 99% del tempo, con solo l'1% in attesa (*Waiting*). Il dato è riferito al frame selezionato e potrebbe includere parte dei tempi di attesa di sincronizzazione (ad esempio il VSync).
- **Impatto dei sistemi nel frame (SH):** *Scripts* 13.795 ms, *GarbageCollector* 9.384 ms, *Rendering* 6.236 ms.
- **Colli di Bottiglia (CH):** il 41% dei frame dell'intera cattura ha un tempo di CPU oltre il budget (16.67 ms; il frame time totale, invece, lo supera in ogni frame), mentre la GPU lo supera nel 20% dei casi.

- **Distribuzione del Frame Time (CH):** minimo 19.198 ms (frame 712); mediana 33.408 ms (frame 156), pari a circa 29.9 FPS e rappresentativa della frequenza tipica dell'applicazione; massimo 262.386 ms, registrato al frame 716, equivalente ad un blocco visibile di oltre un quarto di secondo. In questo frame il marker con il maggior tempo proprio (*self time*, definito più sotto) è il rendering della camera della URP (`UniversalRenderPipeline.RenderSingleCameraInternal`) e non il Garbage Collector.

- **Self Time (SH e CH):** misura il tempo netto speso in una funzione, escluse le sotto-funzioni chiamate. I principali contributi nel frame 808 sono:

  1. `GC.Collect`: raccolta dei rifiuti sul Managed Heap, principale fattore di degrado temporaneo. Nel frame analizzato assorbe 9.290 ms ($\approx$ 28% del tempo di frame), circa il doppio della mediana delle raccolte della cattura (4.567 ms); sull'intera sessione il massimo raggiunge 11.433 ms.
  2. `Gfx.PresentFrame`: richiesta al sistema operativo e al display di mostrare il frame renderizzato. Assorbe 5.192 ms, probabilmente legati alla sincronizzazione della presentazione (es. attesa del VSync).
  3. Routine AR Foundation (`ARSession` e `ARCameraManager`): 7.179 ms complessivi per l'elaborazione dei flussi video e dei sensori di tracciamento.

- **Mean Time (CH):** tempo medio occupato dai sistemi; i più rilevanti sono:

  – `VSync` (16.039 ms): tecnologia che previene lo *screen tearing* (la comparsa contemporanea di porzioni di fotogrammi diversi, visibile come interruzione orizzontale), obbligando la scheda grafica ad attendere il termine del ciclo di aggiornamento del display prima di inviare il fotogramma successivo. Nel profiler è tempo di attesa, non di lavoro.

  – `Scripts` (8.164 ms): codice e logiche eseguite dalla CPU ad ogni frame. Un'elevata complessità ne aumenta la durata, ritardando il fotogramma successivo.

  – `Rendering` (6.866 ms): fase in cui la CPU prepara i comandi grafici e la GPU elabora i dati geometrici e di illuminazione per generare i pixel dell'immagine; scene complesse, shader (programmi eseguiti dalla GPU per calcolare l'aspetto delle superfici) avanzati o risoluzioni elevate ne aumentano la latenza.

  In media, il lavoro effettivo (*Scripts* più *Rendering*, circa 15.0 ms) rientra nel budget di 16.67 ms, mentre la quota maggiore del frame time medio è attesa di VSync. Il frame time di circa 31 ms dipende quindi in larga parte dalla cadenza a 30 FPS e non solo dal carico computazionale; i picchi come `GC.Collect` possono invece spiegare i frame che superano il budget.

5.3.3 Limiti della Sperimentazione

I risultati vanno letti tenendo conto dei seguenti limiti:

- **Build di debug con Profiler collegato:** introducono un overhead aggiuntivo (il Profiler stesso occupa 36.9 MB di memoria); i valori assoluti sono quindi indicativi e, con ogni probabilità, peggiori rispetto a una build di rilascio.
- **Singolo dispositivo:** i test sono stati eseguiti solo sul Motorola Edge 70, quindi i risultati non sono generalizzabili ad altri modelli o fasce di mercato.
- **Frame selezionato:** l'analisi di dettaglio riguarda un singolo frame (il 808), prossimo alla mediana, che non cattura la variabilità della sessione né il caso peggiore (frame 716); per il comportamento complessivo si fa riferimento ai *Capture Highlights*.
- **Misura del tempo GPU:** il *GPU Time* non è disponibile per il frame analizzato, quindi il carico grafico è stimato indirettamente.
- **Limite alla frequenza di aggiornamento:** il tempo mediano di circa 33 ms potrebbe essere in parte determinato da un cap a 30 FPS (VSync, `Application.targetFrameRate` o cadenza del flusso della fotocamera) e non solo dal carico computazionale.

Capitolo 6

Conclusioni

Il confronto tra le due architetture non ha portato a un vincitore assoluto: nel corso del capitolo sono stati analizzati vantaggi e svantaggi di entrambi gli approcci, e la scelta dipende dagli obiettivi dello sviluppatore e dal tipo di applicazione da realizzare e distribuire.

AR Foundation è la soluzione più adatta quando il team di sviluppo è ridotto, l'applicazione deve essere distribuita su più piattaforme, i tempi di produzione devono essere rapidi e si accettano prestazioni comunque buone. Se invece sono prioritari il massimo delle prestazioni, l'accesso alle funzionalità più recenti e il pieno controllo dell'hardware, con un'integrazione completa con Android o iOS, la scelta più appropriata è l'uso diretto di ARCore o ARKit.

Inoltre, l'app sviluppata si presta a diversi ambiti applicativi: l'**arredamento e l'interior design**, per visualizzare mobili e complementi nello spazio domestico prima dell'acquisto; l'**e-commerce**, per anteprime in scala reale di prodotti; l'**architettura e l'edilizia**, per la presentazione di modelli e progetti sul sito; la **didattica e la divulgazione**, per esplorare modelli di oggetti, reperti o strutture anatomiche; e l'**intrattenimento**, ad esempio in giochi o esperienze interattive in realtà aumentata.

L'analisi condotta con l'*Unity Profiler on-device* mostra che l'applicazione non raggiunge il target di 60 FPS: il tempo mediano di frame è 33.408 ms (circa 30 FPS), il minimo è 19.198 ms e si registrano picchi isolati fino a 262.386 ms, dovuti al rendering della camera della URP. La CPU è il limite più frequente (41% dei frame oltre il budget, contro il 20% della GPU), ma il lavoro effettivo medio (*Scripts* e *Rendering*, circa 15.0 ms) rientra nel budget di 16.67 ms, mentre la quota maggiore del frame time medio è attesa di VSync (16.039 ms): il valore mediano è quindi in larga parte riconducibile a una cadenza di 30 FPS, da verificare, e non solo al carico computazionale. Il carico grafico, stimato indirettamente perché il *GPU Time* non è disponibile per il frame analizzato, risulta contenuto (42 *SetPass Calls*, 7 chiamate di disegno in 4 batch, circa 22k triangoli, 46.3 MB di texture), così come l'occupazione di memoria (0.66 GB committed). Le criticità derivano dal costo del tracciamento

AR, che nel frame analizzato assorbe quasi tutto il tempo degli `Update()` (17.07 su 18.33 ms), e dalle allocazioni continue sul *Managed Heap*, in gran parte originate da routine di AR Foundation e del rendering, che attivano il *Garbage Collector*: questo assorbe fino a 11.433 ms e causa buona parte dei frame oltre il budget.

Come sviluppi futuri si individuano diverse direzioni di ottimizzazione. In primo luogo, la verifica del frame rate target e del cap (limite al frame rate che l'applicazione stessa si impone; sarebbe Application.targetFrameRate (insieme a vSyncCount)) a 30 FPS: il target è la frequenza a cui l'applicazione dovrebbe girare e determina il tempo massimo per produrre un frame (circa 33,3 ms a 30 FPS), per cui un frame time vicino a tale valore non indica necessariamente una CPU satura, ma può riflettere un'attesa imposta dal limite. In secondo luogo, l'identificazione, tramite la vista *Hierarchy* del Profiler ordinata per *GC Alloc*, dell'origine delle allocazioni, cioè delle richieste di memoria sul *Managed Heap*, l'area gestita automaticamente per il codice C#; dove queste dipendono dal codice dell'applicazione, possono essere ridotte mediante *caching* dei risultati di operazioni costose, ovvero salvarli in una variabile e riusarli, *object pooling*, ovvero riutilizzo di oggetti attivandoli e disattivandoli al posto di `Instantiate` e `Destroy` e riuso di *buffer*, ad esempio la lista dei risultati dei raycast. Inoltre si può valutare l'introduzione del Garbage Collector incrementale, che distribuisce su più frame la liberazione della memoria inutilizzata, riducendo i picchi di frame time a fronte di un lieve overhead. Un ulteriore intervento riguarda la riduzione del costo del tracciamento AR, ossia dell'elaborazione con cui ARCore stima la posa del dispositivo e analizza l'ambiente, ad esempio disattivando le funzionalità non necessarie come il point cloud, la stima della luce o l'occlusione, tenendo conto del compromesso con la robustezza dell'esperienza d'uso. Infine, una nuova campagna di test con build di rilascio su più dispositivi permetterebbe di ottenere misure più vicine all'esperienza reale, poiché la build di debug con Profiler collegato introduce overhead, e di quantificare i miglioramenti confrontando, prima e dopo gli interventi, frametime e memoria utilizzata.

Bibliografia

[1] Wikipedia. Augmented reality. https://it.wikipedia.org/wiki/Realt%C3%A0_aumentata.

[2] IBM. Augmented reality. https://www.ibm.com/it-it/think/topics/augmented-reality.

[3] pikkart. Storia della realtà aumentata dal 1968 ad oggi. https://www.pikkart.com/servizi/notizie/notizie_fase02.aspx?ID=3464&categoriaVisualizzata=19.

[4] computationalculture. ubiquitous-computing. https://computationalculture.net/from-wimp-to-atlas-rhetorical-figures-of-ubiquitous-computing/.

[5] educause. ubiquitous-computing. https://er.educause.edu/articles/2005/1/pervasive-computing-goes-to-school.

[6] Stanford. ubiquitous-computing. https://www.lri.fr/~mbl/Stanford/CS477/papers/Weiser-SciAm.pdf.

[7] idxf. ar vs other. https://ixdf.org/literature/article/beyond-ar-vs-vr-what-is-the-difference-between-ar-vs-mr-vs-vr-vs-xr.

[8] Stanford. The virtuality continuum. https://displaydaily.com/was-this-the-year-of-ar-vr/.

[9] Unity. Ar foundation. https://docs.unity3d.com/Packages/com.unity.xr.arfoundation@6.6/manual/index.html.

[10] developers.google. Ar foundation. https://developers.google.com/ar/develop/unity-arf/getting-started-ar-foundation?hl=it.

[11] andreasjakl. Ar foundation. https://www.andreasjakl.com/ar-foundation-fundamentals-with-unity-part-1/.

[12] angry-shark studio. Ar foundation vs arcore. https://www.angry-shark-studio.com/blog/ar-foundation-vs-arcore-comparison/.

[13] Wikipedia. Sdk. https://it.wikipedia.org/wiki/Software_development_kit.

[14] Wikipedia. Sdk vs ndk. https://medium.com/@jananirave19/sdk-vs-ndk-in-android-development-whats-the-difference-6098012db7e6.

[15] Wikipedia. Unity. https://it.wikipedia.org/wiki/Unity_(motore_grafico)#Descrizione.

[16] Unity. Unity. https://docs.unity3d.com/540/Documentation/Manual/2Dor3D.html.

[17] Unity. Unity. https://docs.unity3d.com/6000.6/Documentation/ScriptReference/.

[18] Unity. Unity. https://docs.unity.com/en-us/engine/6000.6/manual/.

[19] impulsegamer. Invizimals. https://www.impulsegamer.com/pspinvizimalsthelosttribes.html.

[20] Wikipedia. Google glass. https://it.wikipedia.org/wiki/Google_Glass.

[21] TopMovieClips. Tony stark discovers a new element scene - iron-man 2 (2010) movie clip hd. https://www.youtube.com/watch?v=Ddk9ci6geSs.

[22] trace3D. Slam. https://www.trace3d.app/ar-glossary/slam.

[23] geeksforgeeks. Ar architecture. https://www.geeksforgeeks.org/computer-graphics/architecture-of-augmented-reality-and-its-components/.

[24] credera. Ar foundation architecture. https://credera.com/en-us/insights/the-future-of-mobile-ar-a-comparison-between-arcore-arkit-ar-foundation-and-vuforia.

[25] Unity. Profiler introduction. https://docs.unity3d.com/Manual/profiler-introduction.html.